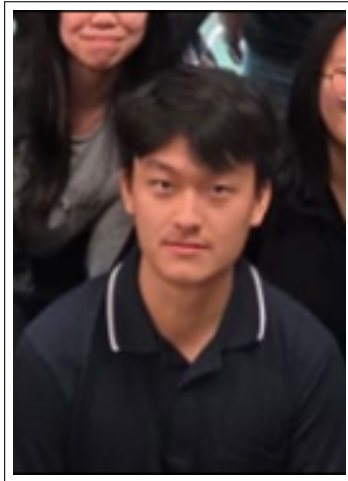


**TK CIPHER BLOCK CIPHER: DESAIN, IMPLEMENTASI, DAN
ANALISIS KEAMANAN**

Laporan Tugas 2

Disusun untuk memenuhi tugas mata kuliah Kriptografi pada Semester 7 (tujuh)
Tahun Akademik 2026/2027

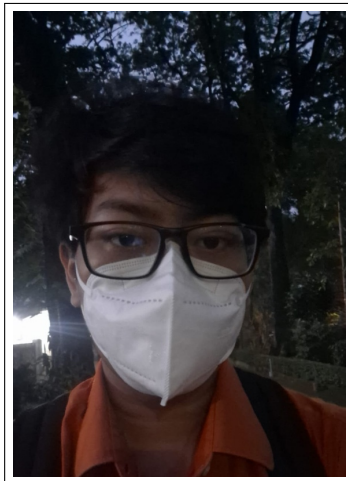
Disusun Oleh:



Brian Ricardo Tamin
(13523126)



Theo Kurniady
(13523154)



Arlow Emmanuel Hergara
(13523161)



Filbert Engyo
(13523163)

**PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
BANDUNG
2026**

LEMBAR PERNYATAAN KEASLIAN

Kami yang bertanda tangan di bawah ini menyatakan bahwa laporan, program, dan seluruh bagian kriptografi pada Tugas 2 IF4020 Kriptografi ini, yaitu cipher TK-Cipher, mode operasi, padding, penurunan key (KDF), dan MAC, adalah hasil karya kami sendiri. Kami tidak menyalin implementasi AES, DES, Blowfish, atau cipher lain, tidak memakai S-box AES, dan tidak memakai library kriptografi pihak ketiga. Sumber yang menjadi inspirasi dicantumkan pada bagian Referensi.

Bandung, 9 Oktober 2026

Nama	NIM	Tanda Tangan
Brian Ricardo Tamin	13523126	
Theo Kurniady	13523154	
Arlow Emmanuel Hergara	13523161	
Filbert Engyo	13523163	

DAFTAR ISI

BAB I	PENDAHULUAN	5
1.1	Latar Belakang	5
1.2	Tujuan dan Ruang Lingkup	5
1.3	Review Cipher Sejenis	5
1.4	Gagasan Utama Rancangan	6
1.5	Sistematika Laporan	6
BAB II	DASAR TEORI	7
2.1	Block Cipher, Confusion, dan Diffusion	7
2.2	Struktur SPN dan Feistel	7
2.3	Komponen Cipher	7
2.3.1	S-box, Differential Uniformity, dan Nonlinearity	7
2.3.2	Permutasi, Operasi Tambahan, Key Schedule, dan Whitening	8
2.4	Mode Operasi	8
2.5	Padding PKCS#7	8
2.6	Integritas Data	8
2.6.1	MAC dan Encrypt-then-MAC	8
2.6.2	CMAC	8
2.6.3	Constant-time Compare dan Padding Oracle	9
2.7	Metrik Analisis	9
2.8	Karya Terkait	9
BAB III	PERANCANGAN BLOCK CIPHER	11
3.1	Gambaran Umum dan Parameter	11
3.2	Representasi State	13
3.3	Struktur Satu Ronde	14
3.3.1	AddRoundKey	14
3.3.2	DiagonalTranspose	14
3.3.3	DynamicSub	15
3.3.4	RowRotator	15
3.3.5	ColumnCascade	15
3.3.6	Alasan Kombinasi Operasi	15
3.4	Alur Enkripsi dan Dekripsi Blok	16
3.5	S-box Dinamis	17
3.5.1	PRNG TKRand	17
3.5.2	Shuffle dan Penghilangan Fixed Point	18
3.5.3	Filter Kualitas	18
3.6	Key Schedule	18
3.7	Mode Operasi	19
3.8	Padding	19
3.9	Penurunan Key dan MAC	21
3.9.1	KDF	21
3.9.2	CMAC-TK	21
3.10	Format File Ciphertext	22

3.11 Rancangan Program	23
3.12 Perbedaan dengan AES	24
3.13 Pelajaran dari Rancangan Awal	24
BAB IV IMPLEMENTASI DAN SIMULASI	25
4.1 Deskripsi Program.....	25
4.1.1 Teknologi dan Dependensi	25
4.1.2 Struktur Repositori dan Modul	25
4.1.3 Cara Menjalankan	26
4.1.4 Dokumentasi API	26
4.2 Hasil Enkripsi dan Dekripsi	27
4.3 Bukti Penanganan Padding.....	28
4.4 Bukti Verifikasi Integritas	28
4.5 Test Vector dan Jejak Satu Blok	29
4.6 Pengujian Otomatis.....	30
BAB V ANALISIS KEAMANAN	31
5.1 Avalanche Effect	31
5.1.1 Flip 1 Bit Plaintext	31
5.1.2 Flip 1 Bit Key	31
5.2 Round Diffusion dan Justifikasi 16 Ronde	32
5.3 Entropi dan Uji Chi-square.....	33
5.4 Histogram Frekuensi Byte	34
5.5 Statistik S-box	38
5.6 Kinerja: Berkas Kecil, Sedang, dan Besar	39
5.7 Pembahasan dan Keterbatasan	40
BAB VI KESIMPULAN DAN SARAN.....	41
6.1 Kesimpulan	41
6.2 Saran	41
REFERENSI.....	42
LAMPIRAN	43
Lampiran A. Tautan Proyek	43
Lampiran B. Pembagian Tugas	43
Lampiran C. Data Avalanche Plaintext per Blok	44
Lampiran D. Perintah Reproduksi Analisis	45

BAB I

PENDAHULUAN

1.1 Latar Belakang

Kerahasiaan dan keutuhan data adalah dua kebutuhan dasar dalam komunikasi digital. Kerahasiaan dijaga dengan enkripsi, sedangkan keutuhan dijaga dengan mekanisme autentikasi pesan. Pada enkripsi simetris, pekerjaan utama dilakukan oleh *block cipher*, yaitu fungsi yang mengubah satu blok data berukuran tetap menjadi blok lain dengan bantuan sebuah kunci. Block cipher kemudian dibungkus oleh mode operasi, padding, dan MAC agar bisa dipakai untuk data berukuran bebas dan tetap aman dari modifikasi.

Tugas 2 mata kuliah IF4020 Kriptografi meminta kami merancang sebuah block cipher baru dari nol, membangun program enkripsi dan dekripsi file di atasnya, lalu menganalisis keamanannya secara empiris. Seluruh bagian kriptografi harus ditulis sendiri. Library kriptografi pihak ketiga dan penyalinan implementasi AES, DES, atau Blowfish tidak diperbolehkan.

Kami menamai hasil rancangan ini TK-Cipher. Cipher ini dibuat untuk tujuan pembelajaran. Ia belum melalui kriptanalisis formal, sehingga tidak dimaksudkan untuk menggantikan cipher standar seperti AES pada sistem nyata.

1.2 Tujuan dan Ruang Lingkup

Tujuan pengerjaan tugas ini adalah sebagai berikut.

1. Merancang block cipher berstruktur SPN dengan blok 128 bit, key 128, 192, atau 256 bit, S-box yang dirancang dan didokumentasikan sendiri, transposisi, minimal satu operasi tambahan, dan round key unik per ronde.
2. Mengimplementasikan lima mode operasi (ECB, CBC, CFB, OFB, dan CTR), padding PKCS#7, penurunan key (KDF), serta integritas Encrypt-then-MAC dengan MAC buatan sendiri.
3. Membuat program enkripsi dan dekripsi file teks maupun biner dengan format file yang mendeskripsikan dirinya sendiri, yang berjalan di Linux dan Windows.
4. Menganalisis keamanan cipher melalui avalanche effect, entropi, histogram, uji chi-square, statistik S-box, dan benchmark di kelima mode.

Ruang lingkup mencakup cipher, mode, padding, KDF, MAC, format file, antarmuka baris perintah (CLI), skrip analisis, dokumentasi API, dan executable untuk Linux dan Windows. Hal yang berada di luar ruang lingkup adalah antarmuka grafis, mode AEAD lain seperti GCM, penurunan key dari kata sandi, dan pemrosesan file yang lebih besar dari memori.

1.3 Review Cipher Sejenis

Sebelum merancang TK-Cipher, kami meninjau beberapa block cipher yang ada untuk memahami pilihan struktur dan komponennya. Tabel 1.1 merangkum cipher yang ditinjau.

Tabel 1.1: Perbandingan TK-Cipher dengan cipher sejenis

Cipher	Blok	Key	Ronde	Struktur	Komponen utama
DES [1, 2]	64	56	16	Feistel	8 S-box 6 ke 4 bit tetap
AES [3]	128	128/192/256	10/12/14	SPN	S-box 8 bit tetap, ShiftRows, MixColumns
XTEA	64	128	64	Feistel	ARX, tanpa S-box
PRESENT	64	80/128	31	SPN	S-box 4 bit, permutasi bit
RC6 [4]	128	128/192/256	20	Feistel ala RC5	Rotasi bergantung data, perkalian
Twofish	128	hingga 256	16	Feistel	S-box bergantung key, MDS, PHT
TK-Cipher	128	128/192/256	16 + whitening	SPN	S-box bergantung key, transpose, rotasi bit, penjumlahan modular

Beberapa kesamaan dan perbedaan dengan cipher tersebut penting sebagai pembanding rancangan kami. Tidak ada implementasi yang disalin.

- AES memiliki state matriks 4x4 byte dan kerangka SPN, yang juga dimiliki TK-Cipher. Komponennya berbeda total, yaitu tanpa Galois Field dan tanpa S-box statis (Subbab 3.12).
- Twofish juga memakai S-box yang bergantung pada key. TK-Cipher membangkitkannya dengan cara berbeda, yaitu mengacak permutasi dengan PRNG buatan sendiri lalu menyaringnya dengan ukuran kualitas.
- XTEA menunjukkan bahwa penjumlahan modular, rotasi, dan XOR (ARX) sudah cukup untuk membuat komponen sederhana yang non-linear. Konsep yang sama dipakai pada PRNG TKRand dan operasi ColumnCascade.
- PRESENT memakai lapisan permutasi bit yang sederhana untuk menyebarkan perubahan antar S-box. Peran serupa dijalankan oleh RowRotator pada TK-Cipher.

1.4 Gagasan Utama Rancangan

TK-Cipher adalah SPN byte-level dengan state matriks 4x4 yang menjalankan 16 ronde dan satu whitening akhir. Ide utamanya sebagai berikut.

1. S-box dinamis. S-box dibangkitkan dari key lewat PRNG ARX buatan sendiri (TKRand) dan Fisher-Yates shuffle, lalu disaring dengan batas differential uniformity dan nonlinearity. Tiap key punya S-box sendiri.
2. Transposisi penuh. Operasi DiagonalTranspose menukar baris dan kolom state, sehingga operasi per baris dan per kolom pada ronde berikutnya saling bergantian arah.
3. Dua operasi tambahan. RowRotator merotasi bit di tiap baris melewati batas byte, dan ColumnCascade menjumlahkan byte dalam kolom secara berantai (mod 256) sehingga carry menambah sifat non-linear.
4. Program lengkap. Di atas cipher dibangun lima mode, padding PKCS#7, KDF, CMAC buatan sendiri, dan format file yang melindungi header sekaligus ciphertext dengan MAC.

1.5 Sistematika Laporan

Laporan ini terdiri dari enam bab. Bab I berisi latar belakang, tujuan, tinjauan cipher sejenis, dan gagasan utama. Bab II memuat dasar teori. Bab III menjelaskan perancangan TK-Cipher beserta diagramnya. Bab IV memaparkan implementasi program dan hasil simulasi, termasuk bukti padding dan verifikasi integritas. Bab V berisi analisis keamanan. Bab VI berisi kesimpulan dan saran. Setelah itu terdapat daftar referensi dan lampiran.

BAB II

DASAR TEORI

2.1 Block Cipher, Confusion, dan Diffusion

Block cipher adalah fungsi $E_K : \{0, 1\}^n \rightarrow \{0, 1\}^n$ yang bijektif untuk setiap key K , sehingga ada fungsi kebalikan D_K dengan $D_K(E_K(P)) = P$. Cipher yang aman harus berperilaku seperti permutasi acak bagi pihak yang tidak mengetahui key. Shannon [1] merumuskan dua sifat yang harus dimiliki rancangan cipher.

- Confusion, yaitu hubungan antara key dan ciphertext dibuat serumit mungkin, biasanya lewat komponen non-linear seperti S-box.
- Diffusion, yaitu pengaruh satu bit plaintext atau key disebarkan ke banyak bit ciphertext. Kriteria praktisnya adalah *avalanche effect* [2], di mana mengubah satu bit input mengubah sekitar separuh bit output.

Cipher modern memperolehnya dengan mengulang sebuah fungsi ronde beberapa kali (*iterated cipher*), dengan round key berbeda tiap ronde yang diturunkan dari master key melalui *key schedule*.

2.2 Struktur SPN dan Feistel

Dua struktur yang paling umum adalah jaringan Feistel [1] dan *Substitution-Permutation Network* (SPN).

Feistel membagi blok menjadi dua bagian. Pada tiap ronde, satu bagian diubah dengan XOR terhadap keluaran fungsi F atas bagian lainnya, lalu kedua bagian ditukar. Karena strukturnya, F tidak harus invertible, tetapi hanya separuh blok yang berubah per ronde sehingga diffusion lebih lambat.

SPN mengubah seluruh blok pada tiap ronde lewat lapisan substitusi (S-box) dan lapisan permutasi atau pencampuran linear. Seluruh bit terlibat di setiap ronde sehingga diffusion cepat, tetapi setiap komponen harus invertible agar dekripsi mungkin. AES adalah contoh SPN yang paling dikenal. TK-Cipher memilih SPN karena S-box yang berupa permutasi 0 sampai 255 secara alami bisa dibalik.

2.3 Komponen Cipher

2.3.1 S-box, Differential Uniformity, dan Nonlinearity

S-box 8 bit adalah tabel permutasi $S : \{0, 1\}^8 \rightarrow \{0, 1\}^8$ yang menjadi sumber utama sifat non-linear. Kualitasnya dinilai dengan dua ukuran.

- **Differential Uniformity (DU)**

Differential uniformity adalah jumlah maksimum x yang memenuhi $S(x) \oplus S(x \oplus a) = b$ untuk $a \neq 0$:

$$DU(S) = \max_{a \neq 0, b} |\{x : S(x) \oplus S(x \oplus a) = b\}|.$$

Nilai kecil berarti S-box tahan terhadap kriptanalisis diferensial. Nilai terkecil yang mungkin adalah 2, sedangkan fungsi identitas bernilai 256.

- **Nonlinearity (NL)**

Nonlinearity mengukur jarak S-box dari fungsi affine. Dengan transformasi Walsh-Hadamard $W_S(a, b) = \sum_x (-1)^{a \cdot x \oplus b \cdot S(x)}$, nilainya

$$NL(S) = 128 - \frac{1}{2} \max_{a, b \neq 0} |W_S(a, b)|.$$

Nilai besar berarti tahan terhadap kriptanalisis linear. Batas atas untuk S-box 8 bit adalah 112, sedangkan fungsi identitas bernilai 0. Sebagai pembandingan, S-box AES memiliki DU 4 dan NL 112.

2.3.2 Permutasi, Operasi Tambahan, Key Schedule, dan Whitening

Permutasi atau transposisi memindahkan posisi byte atau bit tanpa mengubah nilainya, sehingga menjadi sarana diffusion antar S-box. Operasi tambahan seperti rotasi bit dan penjumlahan modular membantu pencampuran. Penjumlahan modular bersifat non-linear terhadap XOR karena adanya carry. Key schedule menurunkan sejumlah round key dari master key. Rancangan yang baik membuat round key berbeda untuk tiap ronde dan sulit ditelusuri ke master key. Whitening adalah XOR dengan sebuah key tambahan di awal atau akhir cipher sehingga lapisan terluar cipher tidak bisa dikupas tanpa mengetahui key.

2.4 Mode Operasi

Block cipher hanya memproses satu blok. Mode operasi [1] menentukan cara memakainya untuk data panjang. Misalkan E enkripsi blok, D dekripsi blok, P_i blok plaintext, dan C_i blok ciphertext.

Tabel 2.1: Lima mode operasi

Mode	Enkripsi	Dekripsi	Catatan
ECB	$C_i = E(P_i)$	$P_i = D(C_i)$	Blok sama menghasilkan ciphertext sama, pola bocor
CBC	$C_i = E(P_i \oplus C_{i-1})$	$P_i = D(C_i) \oplus C_{i-1}$	$C_{-1} = IV$, enkripsi serial
CFB	$C_i = P_i \oplus E(C_{i-1})$	$P_i = C_i \oplus E(C_{i-1})$	$C_{-1} = IV$, hanya memakai E
OFB	$O_i = E(O_{i-1})$, $C_i = P_i \oplus O_i$	sama	$O_{-1} = IV$, keystream tidak bergantung plaintext
CTR	$C_i = P_i \oplus E(CTR_0 + i)$	sama	Counter 128 bit, dapat diparalelkan

ECB tidak butuh IV. CBC, CFB, dan OFB butuh IV yang tidak boleh diulang pada key yang sama. CTR butuh nilai counter awal. Pada OFB dan CTR, pemakaian ulang IV atau counter dengan key yang sama menghasilkan keystream yang sama sehingga XOR dua ciphertext membocorkan XOR dua plaintext.

2.5 Padding PKCS#7

Mode blok (ECB dan CBC) membutuhkan data yang panjangnya kelipatan ukuran blok. PKCS#7 menambahkan $p = 16 - (|M| \bmod 16)$ byte yang masing-masing bernilai p , sehingga $1 \leq p \leq 16$. Data yang sudah pas kelipatan 16 tetap ditambah satu blok penuh, sehingga byte terakhir selalu menunjukkan jumlah padding tanpa ambigu.

2.6 Integritas Data

2.6.1 MAC dan Encrypt-then-MAC

Enkripsi saja tidak menjamin keutuhan. Penyerang yang mengubah ciphertext bisa menyebabkan perubahan plaintext yang terprediksi, terutama di mode stream seperti OFB dan CTR. *Message Authentication Code* (MAC) menghasilkan tag berkunci atas pesan, sehingga perubahan apa pun terdeteksi. Pada konstruksi Encrypt-then-MAC, pesan dienkripsi lebih dulu lalu MAC dihitung atas ciphertext. Bellare dan Namprempre menunjukkan konstruksi ini aman secara umum. Keuntungan praktisnya, tag bisa diperiksa sebelum dekripsi sehingga data yang dimodifikasi tidak pernah diproses.

2.6.2 CMAC

CMAC (OMAC1) adalah MAC berbasis block cipher yang memperbaiki kelemahan CBC-MAC pada pesan berbeda panjang. Dari $L = E_K(0^{128})$ diturunkan dua subkey $K_1 = \text{dbl}(L)$ dan $K_2 = \text{dbl}(K_1)$, dengan dbl

adalah perkalian dengan x di $GF(2^{128})$. Blok terakhir yang penuh di-XOR dengan K_1 , sedangkan blok tak penuh diberi padding $80\ 00\ \dots$ lalu di-XOR dengan K_2 . Setelah itu seluruh blok dirantai seperti CBC dan keluaran terakhir menjadi tag.

2.6.3 Constant-time Compare dan Padding Oracle

Membandingkan tag dengan operasi yang berhenti pada byte pertama yang berbeda membocorkan lewat waktu eksekusi seberapa panjang awalan yang benar. Perbandingan *constant-time* memeriksa semua byte tanpa percabangan yang bergantung data. Selain itu, pesan kesalahan padding yang bisa dibedakan dari kesalahan lain dapat dipakai untuk menebak plaintext (*padding oracle*). Pemeriksaan MAC sebelum dekripsi menutup celah ini.

2.7 Metrik Analisis

- **Avalanche Effect**

Avalanche effect diukur sebagai persentase bit ciphertext yang berubah ketika satu bit plaintext atau key dibalik. Nilai ideal adalah 50%.

- **Entropi Shannon**

Entropi Shannon per byte untuk distribusi frekuensi p_i atas 256 nilai byte:

$$H = - \sum_{i=0}^{255} p_i \log_2 p_i \quad (\text{maksimum 8 bit/byte}).$$

- **Uji Chi-square**

Uji chi-square menguji keseragaman distribusi byte. Dengan O_i frekuensi teramati dan $E = N/256$ frekuensi harapan,

$$\chi^2 = \sum_{i=0}^{255} \frac{(O_i - E)^2}{E},$$

dengan derajat kebebasan 255. Pada taraf signifikansi $\alpha = 0,05$, data seragam diterima jika $\chi^2 \leq 293,248$. Pada data yang benar-benar acak, sekitar 5% pengujian akan gagal secara kebetulan.

- **Histogram**

Histogram menampilkan frekuensi tiap nilai byte. Ciphertext yang baik memberi histogram yang mendekati rata.

2.8 Karya Terkait

Beberapa cipher yang relevan dengan rancangan kami dikelompokkan menurut gagasan utamanya. Perbandingan parameternya ada pada Tabel 1.1.

- SPN dengan S-box tetap: AES [3] memakai SPN dengan S-box 8 bit tetap, ShiftRows, dan MixColumns di $GF(2^8)$. PRESENT memakai SPN ringan dengan S-box 4 bit dan permutasi bit. Keduanya menunjukkan bahwa lapisan substitusi dan lapisan permutasi yang bergantian cukup untuk diffusion yang cepat.
- Cipher berbasis ARX: XTEA hanya memakai penjumlahan modular, rotasi, dan XOR tanpa S-box. RC6 [4] menambahkan rotasi yang bergantung pada data dan perkalian. Operasi seperti ini sederhana dan mudah dibuat non-linear lewat carry.

- S-box bergantung key: Twofish memakai S-box yang dibentuk dari key sehingga penyerang tidak mengetahui tabel substitusi yang dipakai.
- Teknik kriptanalisis: tutorial Heys menjelaskan kriptanalisis linear dan diferensial yang menjadi dasar ukuran DU dan NL pada Subbab 2.3.1.

BAB III

PERANCANGAN BLOCK CIPHER

Bab ini menjelaskan rancangan TK-Cipher beserta alasan tiap pilihannya. Penjelasan lengkap ada pada docs/DESIGN.md, dan diagram aslinya ada di docs/diagrams/ pada repositori.

3.1 Gambaran Umum dan Parameter

TK-Cipher adalah block cipher berstruktur SPN yang bekerja di level byte dengan state matriks 4x4. Tabel 3.1 merangkum parameter rancangan dibandingkan ketentuan spesifikasi tugas.

Tabel 3.1: Parameter TK-Cipher

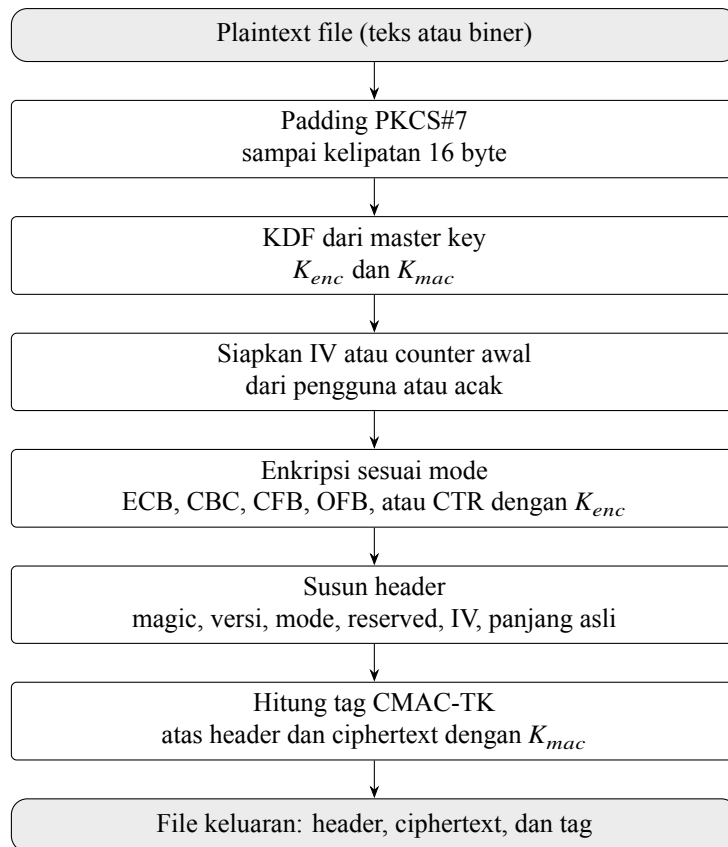
Item	TK-Cipher	Ketentuan spesifikasi
Ukuran blok	128 bit (16 byte)	Minimal 64 bit
Master key	128, 192, atau 256 bit	Minimal sama dengan ukuran blok
Level operasi	Byte, state matriks 4x4	Bit, byte, atau hex
Struktur	SPN, 16 ronde dan whitening akhir	n ronde dengan justifikasi
Round key	17 key, unik per ronde	Diturunkan dari master key

Komponen wajib dari spesifikasi dipenuhi oleh komponen TK-Cipher berikut.

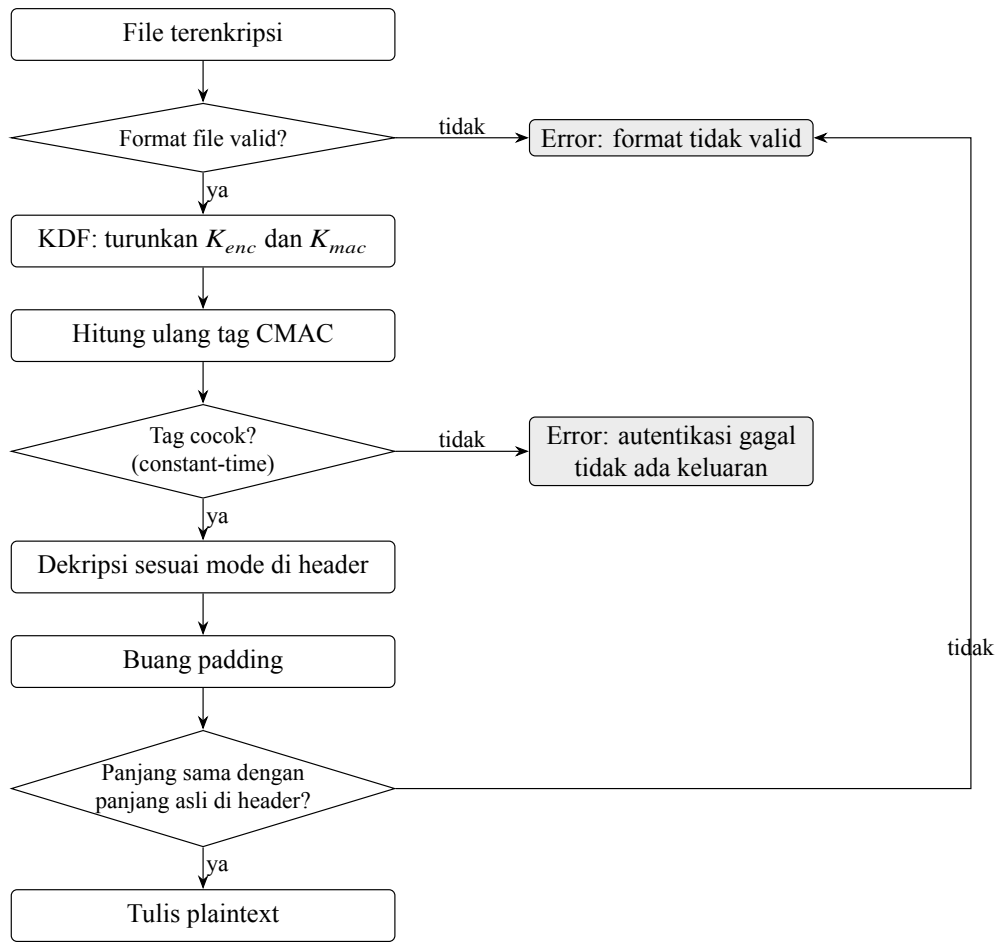
Tabel 3.2: Pemenuhan komponen wajib

Komponen wajib	Komponen TK-Cipher	Keterangan
Substitusi (S-box)	DynamicSub	S-box 256 entri dibangkitkan dari key dan invertible
Transposisi	DiagonalTranspose	Transpose matriks state
Operasi tambahan (minimal 1)	RowRotator dan ColumnCascade	Rotasi bit per baris dan penjumlahan modular per kolom
Round key	AddRoundKey	Round key berbeda tiap ronde dari key schedule

Alur besar program dari plaintext sampai file keluaran ditunjukkan pada Gambar 3.1, dan alur dekripsi pada Gambar 3.2. Pada dekripsi, tag diverifikasi sebelum plaintext dihasilkan.



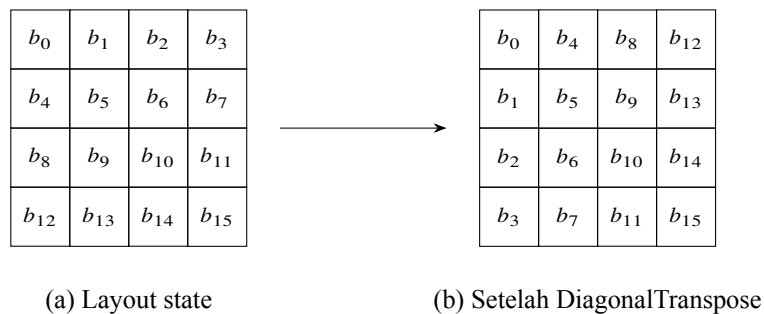
Gambar 3.1: Alur enkripsi file



Gambar 3.2: Alur dekripsi file

3.2 Representasi State

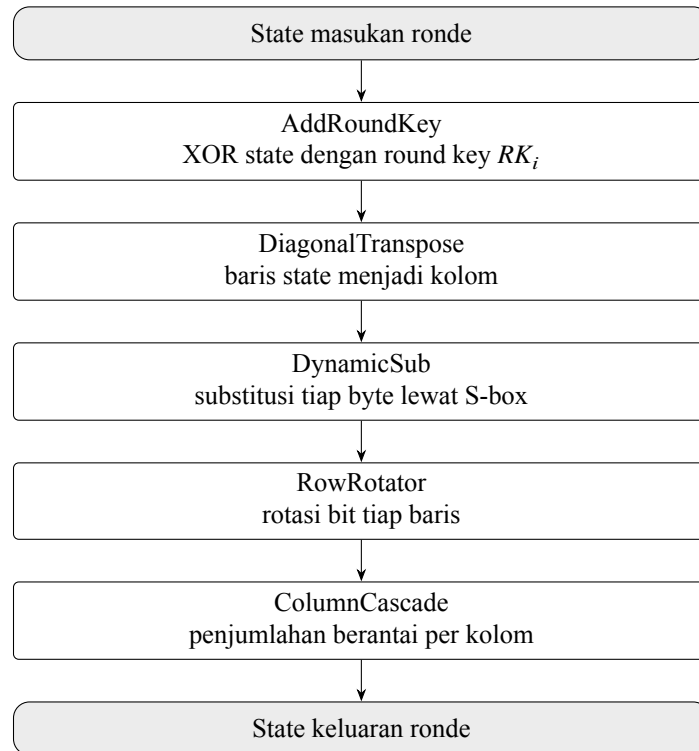
Blok 16 byte $b_0, \dots, b_{15}$ disusun baris demi baris menjadi matriks 4 baris dan 4 kolom, dengan $\text{state}[r][c] = b_{4r+c}$. Round key berukuran 16 byte memakai layout yang sama. Semua operasi ronde bekerja pada matriks ini, sebagian per byte, sebagian per baris, dan sebagian per kolom.



Gambar 3.3: State 4x4 dan efek transpose

3.3 Struktur Satu Ronde

Setiap ronde menjalankan lima operasi berurutan (Gambar 3.4):



Gambar 3.4: Isi satu ronde enkripsi

Tabel 3.3: Operasi dalam satu ronde dan inverse-nya

Operasi	Dikerjakan	Peran	Inverse
AddRoundKey	XOR state dengan round key	Memasukkan key	XOR dengan key yang sama
DiagonalTranspose	Baris menjadi kolom	Transposisi antar baris dan kolom	Dirinya sendiri
DynamicSub	Tiap byte lewat S-box	Substitusi, sumber confusion	S-box inverse
RowRotator	Rotasi bit tiap baris	Menyebarkan bit melewati batas byte	Rotasi ke arah sebaliknya
ColumnCascade	Penjumlahan berantai per kolom	Mencampur byte antar baris	Pengurangan dengan urutan dibalik

3.3.1 AddRoundKey

State di-XOR byte demi byte dengan round key ronde i : $s'_j = s_j \oplus RK_{i,j}$. Operasi ini satu-satunya tempat key masuk ke dalam ronde. Inverse-nya sama dengan operasinya sendiri.

3.3.2 DiagonalTranspose

Operasi ini menukar baris dan kolom: $out[4r + c] = in[4c + r]$. Byte di diagonal utama tetap, byte lain bertukar dengan pasangannya di seberang diagonal (Gambar 3.3). Transpose adalah involusi sehingga inverse-nya sama dengan dirinya. Perannya adalah mengganti arah pencampuran. Pada ronde berikutnya, apa yang tadinya satu baris menjadi satu kolom, sehingga operasi per baris dan per kolom saling bergantian menyebarkan perubahan.

3.3.3 DynamicSub

Setiap byte state diganti dengan $S[b]$, dengan S adalah S-box dinamis hasil pembangkitan dari key (Subbab 3.5). Dekripsi memakai tabel inverse S^{-1} . Operasi ini menjadi sumber utama sifat non-linear dan confusion.

3.3.4 RowRotator

Tiap baris dibaca sebagai word 32 bit big-endian lalu dirotasi bit ke kiri sebanyak $ROT = [1, 3, 5, 7]$ untuk baris 0 sampai 3. Semua baris dirotasi, termasuk baris 0, dengan jumlah ganjil yang berbeda agar bit melintasi batas byte. Inverse-nya adalah rotasi ke kanan dengan jumlah yang sama. Kode 3.1 memperlihatkan implementasinya.

Kode 3.1: Rotasi bit per baris (rounds.py)

```
def _rotate_rows(state, left):
    out = []
    for r in range(4):
        word = int.from_bytes(bytes(state[4*r : 4*r + 4]), "big")
        n = ROT[r] if left else 32 - ROT[r]
        word = ((word << n) | (word >> (32 - n))) & 0xFFFFFFFF
        out.extend(word.to_bytes(4, "big"))
    return out
```

3.3.5 ColumnCascade

Untuk setiap kolom (a, b, d, e) yang berisi baris 0 sampai 3, seluruh penjumlahan dilakukan mod 256:

$$b \leftarrow b + a, \quad d \leftarrow d + b, \quad e \leftarrow e + d, \quad a \leftarrow a + e.$$

Inverse-nya memakai pengurangan dengan urutan dibalik: $a \leftarrow a - e, e \leftarrow e - d, d \leftarrow d - b, b \leftarrow b - a$. Satu byte yang berubah di kolom menjalar ke keempat byte kolom itu. Misalnya kolom $(1, 0, 0, 0)$ menjadi $(2, 1, 1, 1)$. Carry dari penjumlahan menambah sifat non-linear terhadap XOR. Operasi ini mengisi peran MixColumns pada AES tanpa aritmetika Galois Field.

Kode 3.2: ColumnCascade (rounds.py)

```
def column_cascade(state):
    out = list(state)
    for c in range(4):
        a, b, d, e = out[c], out[4 + c], out[8 + c], out[12 + c]
        b = (b + a) & 0xFF
        d = (d + b) & 0xFF
        e = (e + d) & 0xFF
        a = (a + e) & 0xFF
        out[c], out[4 + c], out[8 + c], out[12 + c] = a, b, d, e
    return out
```

3.3.6 Alasan Kombinasi Operasi

Gabungan operasi di atas menghasilkan diffusion dengan jalur berikut. DynamicSub membuat perubahan satu bit memengaruhi seluruh bit di byte itu. RowRotator membawanya ke byte tetangga dalam baris yang sama. ColumnCascade membawanya ke seluruh byte dalam kolom. Pada ronde berikutnya, DiagonalTranspose menukar baris dan kolom sehingga penyebaran berlanjut ke arah lainnya. Pengukuran pada implementasi (Subbab 5.2) menunjukkan perubahan satu bit sudah memengaruhi seluruh blok pada ronde ke-4.

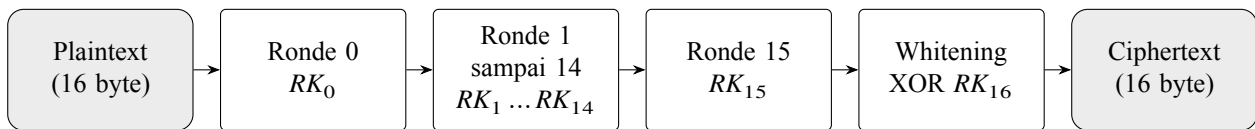
3.4 Alur Enkripsi dan Dekripsi Blok

Enkripsi satu blok menjalankan 16 ronde dengan round key $RK_0, \dots, RK_{15}$, lalu *whitening* berupa XOR dengan RK_{16} (Gambar 3.5).

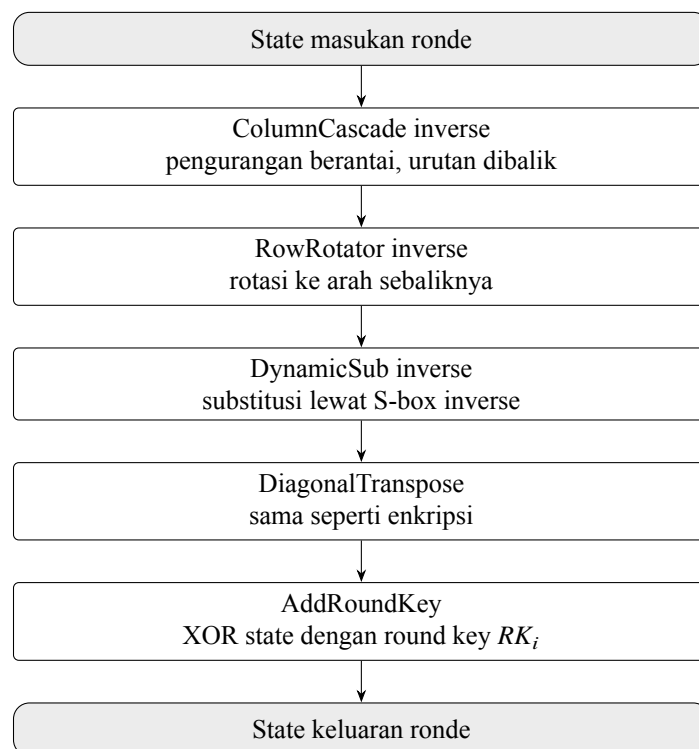
$$\text{Enc}(P) : s \leftarrow P, \text{ untuk } i = 0..15 : s \leftarrow R_i(s), C \leftarrow s \oplus RK_{16},$$

$$\text{Dec}(C) : s \leftarrow C \oplus RK_{16}, \text{ untuk } i = 15..0 : s \leftarrow R_i^{-1}(s), P \leftarrow s.$$

Dekripsi membuang *whitening* lalu menjalankan inverse tiap operasi ronde dalam urutan terbalik: Column-Cascade inverse, RowRotator inverse, S-box inverse, DiagonalTranspose, lalu AddRoundKey (Gambar 3.6). *Whitening* diperlukan karena ronde terakhir berakhir pada operasi tanpa key. Tanpa XOR akhir, lapisan itu bisa dikupas tanpa mengetahui round key. Pada TK-Cipher pengupasan itu masih membutuhkan S-box yang bergantung key, tetapi *whitening* tetap dipertahankan karena murah (satu XOR) dan merupakan praktik rancangan yang umum.



Gambar 3.5: Enkripsi satu blok



Gambar 3.6: Isi satu ronde dekripsi (kebalikan)

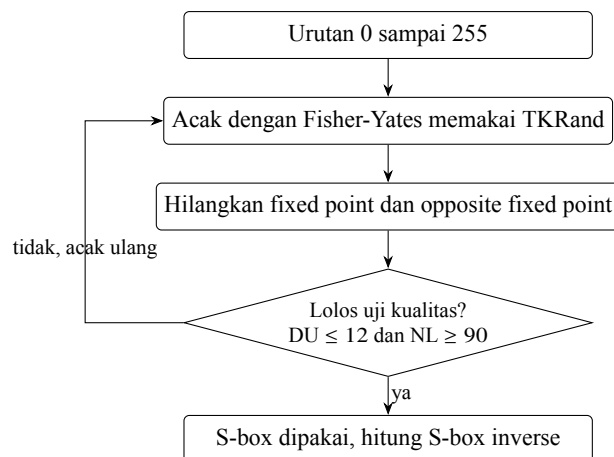
Kode 3.3: Enkripsi blok (cipher.py)

```
def encrypt_block(self, block):
    s = list(block)
    for i in range(self.rounds):
        s = add_round_key(s, self.round_keys[i])
        s = diagonal_transpose(s)
        s = dynamic_sub(s, self.sbox.forward)
        s = row_rotator(s)
        s = column_cascade(s)
    s = add_round_key(s, self.round_keys[self.rounds]) # whitening
    return bytes(s)
```

Jumlah 16 ronde ditentukan dari pengukuran. Perubahan satu bit mencapai diffusion penuh pada ronde ke-4, sehingga 16 ronde memberi margin empat kali lipat (hasil pada Subbab 5.2).

3.5 S-box Dinamis

Berbeda dengan AES yang memakai satu S-box tetap, TK-Cipher membangkitkan S-box dari key. S-box AES tidak dipakai sama sekali. Prosesnya (Gambar 3.7) deterministik, sehingga key yang sama selalu menghasilkan S-box yang sama dan dekripsi dapat membangunnya ulang hanya dari key.



Gambar 3.7: Pembangkitan S-box dinamis

3.5.1 PRNG TKRand

S-box diacak dengan PRNG ARX 4x32 bit buatan sendiri bernama TKRand. Modul random milik Python tidak dipakai.

- State awal: $[G, \text{rotl}(G, 8), \text{rotl}(G, 16), \text{rotl}(G, 24)]$ dengan $G = 0 \times 9\text{E}3779\text{B}9$ (golden ratio).
- Seed adalah "TKC-SBOX" + key + panjang key. Tiap byte seed ditambahkan ke satu word state, lalu state dicampur dengan fungsi mix. Setelah itu dilakukan 32 kali mix sebagai warm-up.
- $\text{mix}(a, b, c, d)$: $a += b$, $d = \text{rotl}(d \oplus a, 13)$, $c += d$, $b = \text{rotl}(b \oplus c, 9)$, $a += b$, $d = \text{rotl}(d \oplus a, 5)$, $c += d$, $b = \text{rotl}(b \oplus c, 11)$.
- Keluaran 32 bit adalah $s_0 \oplus s_2$ setelah mix. Bilangan di rentang $[0, n)$ diambil dengan rejection sampling agar tidak ada bias modulo.

3.5.2 Shuffle dan Penghilangan Fixed Point

Urutan 0 sampai 255 diacak dengan algoritma Fisher-Yates memakai TKRand, sehingga hasilnya pasti permuatasi dan dapat dibalik. Selanjutnya byte yang tidak berubah ($S[x] = x$) dan yang hanya terbalik seluruh bitnya ($S[x] = x \oplus FF$) dihilangkan dengan menukarnya dengan posisi acak lain, diulang sampai tidak tersisa.

3.5.3 Filter Kualitas

Hasil shuffle hanya diterima jika memenuhi dua syarat: differential uniformity ≤ 12 dan nonlinearity ≥ 90 . Jika gagal, shuffle diulang dengan PRNG yang dilanjutkan, sehingga proses tetap deterministik per key. Ambang ini dipilih setelah meninjau statistik pada 600 kandidat sebelum filter (Subbab 5.5). Ambang tersebut membuang ekor terlemah kandidat dengan biaya rata-rata sekitar 1,18 percobaan per key, sedangkan ambang yang lebih ketat menambah waktu pembangkitan secara berarti tanpa perbaikan yang sebanding.

Kode 3.4: Pembangkitan S-box (sbox.py)

```
rng = TKRand(SEED_PREFIX + key + bytes([len(key)]))
while True:
    s = _shuffle(rng) # Fisher-Yates dengan TKRand
    _fix_points(s, rng) # buang S[x]==x dan S[x]==x^0xFF
    du = differential_uniformity(s)
    if du > MAX_DU: # MAX_DU = 12
        continue
    nl = nonlinearity(s)
    if nl >= MIN_NL: # MIN_NL = 90
        break
inv = [0] * 256
for x, v in enumerate(s):
    inv[v] = x # S-box inverse
```

3.6 Key Schedule

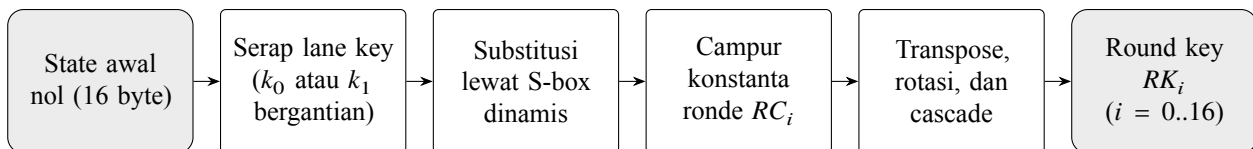
Key schedule menurunkan 17 round key (16 ronde ditambah whitening) dari master key. Rancangannya bergaya sponge, yaitu bagian key diserap bergantian pada tiap langkah (Gambar 3.8). Dua lane key dibentuk dari master key: k_0 adalah 16 byte pertama, sedangkan k_1 bergantung pada ukuran key.

- Key 128 bit: $k_1 = S[\text{rot}_5(k_0)]$, yaitu k_0 dirotasi 5 byte lalu tiap byte disubstitusi S-box.
- Key 192 bit: k_1 adalah 8 byte terakhir key ditambah 8 byte nol.
- Key 256 bit: k_1 adalah 16 byte terakhir key.

Kode 3.5: Key schedule (key_schedule.py, diringkaskan)

```
t = [0] * 16
for i in range(rounds + 1):
    lane = k0 if i % 2 == 0 else k1
    t = [x ^ k for x, k in zip(t, lane)] # serap lane key
    t = dynamic_sub(t, sbox.forward) # non-linear
    rc = round_constant(i).to_bytes(4, "big") # RC_i
    t = [x ^ rc[j % 4] for j, x in enumerate(t)]
    t = column_cascade(row_rotator(diagonal_transpose(t)))
    round_keys.append(bytes(t)) # RK_i
```

Konstanta ronde $RC_i = \text{rotl}_{32}(G, 5i \bmod 32) \oplus (i \cdot 0x01010101)$ membuat tiap langkah berbeda sehingga tidak ada dua round key yang dibangun dengan cara identik. Operasi transpose, rotator, dan cascade dari ronde cipher dipakai ulang agar perubahan satu bit key cepat menyebar ke seluruh round key. Karena S-box dinamis ikut dipakai, hubungan round key dengan master key tidak linear.



Gambar 3.8: Key schedule

3.7 Mode Operasi

Kelima mode ditulis sendiri dengan blok penuh 128 bit. E adalah enkripsi blok TK-Cipher berkunci key enkripsi. Data masuk mode sudah di-pad sehingga panjangnya kelipatan 16 byte. Gambar 3.9 menunjukkan alur ketiga blok pertama tiap mode, dan rumusnya ada pada Tabel 2.1.

Mode CFB, OFB, dan CTR hanya memakai arah enkripsi blok, bahkan untuk dekripsi. IV atau counter awal berasal dari user lewat opsi `--iv` sebanyak 32 karakter hex. Jika tidak diberikan, program membangkitkannya secara acak dengan `os.urandom(16)`. Pada mode CTR, nilai tersebut adalah counter awal dan counter bertambah satu per blok sebagai integer 128 bit big-endian dengan wrap pada 2^{128} . Mode ECB tidak memakai IV, dan opsi `--iv` ditolak untuk ECB.

3.8 Padding

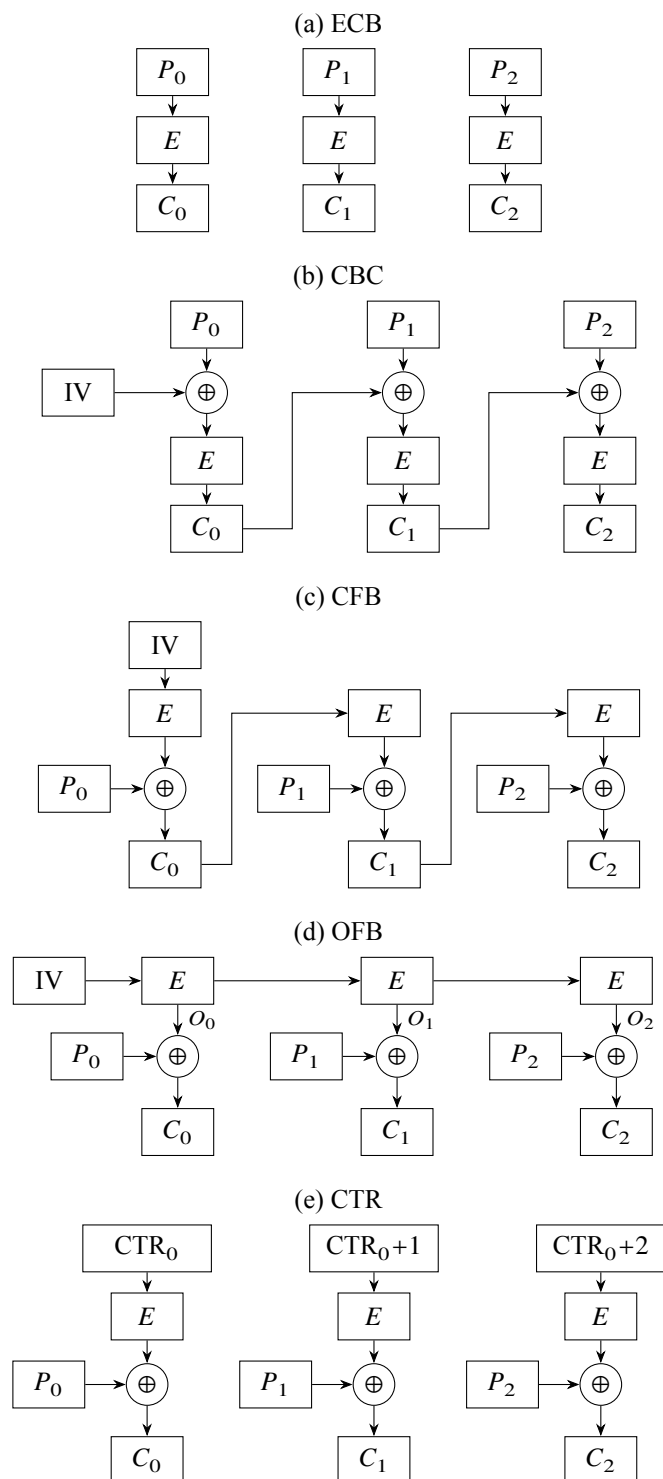
Program memakai PKCS#7 di semua mode. Mekanismenya sesuai Tabel 3.4: tambahkan $p = 16 - (n \bmod 16)$ byte bernilai p , dengan $1 \leq p \leq 16$.

Tabel 3.4: Padding untuk berbagai panjang data

Panjang asli n	Padding p	Panjang setelah padding	Isi padding
0 (kosong)	16	16	16 byte 0x10
1	15	16	15 byte 0x0f
15	1	16	1 byte 0x01
16 (pas)	16	32	16 byte 0x10
17	15	32	15 byte 0x0f

Justifikasi pemilihan PKCS#7 sebagai berikut.

1. Tidak ambigu: padding selalu ada, termasuk saat data pas kelipatan 16. Data yang kebetulan berakhir dengan byte menyerupai padding tetap bisa dibedakan.



Gambar 3.9: Alur lima mode operasi untuk tiga blok pertama

2. Satu jalur kode untuk semua mode: CFB, OFB, dan CTR sebenarnya bersifat stream dan tidak butuh padding. Namun, kami tetap memadnya agar format file, pengujian, dan penanganan edge case seragam di kelima mode.
3. Aturan unpad sederhana dan dapat diverifikasi: fungsi unpad memeriksa bahwa data tidak kosong, panjangnya kelipatan 16, byte terakhir antara 1 dan 16 (bukan nol dan tidak melebihi ukuran blok), dan semua byte padding bernilai sama dengan byte terakhir.
4. Aman dari padding oracle: unpad hanya dijalankan setelah tag MAC valid, sehingga penyerang tidak bisa mengamati kesalahan padding dari file yang dimodifikasi.

Panjang plaintext asli juga disimpan di header file dan dicocokkan setelah unpad sebagai pemeriksaan tambahan.

3.9 Penurunan Key dan MAC

3.9.1 KDF

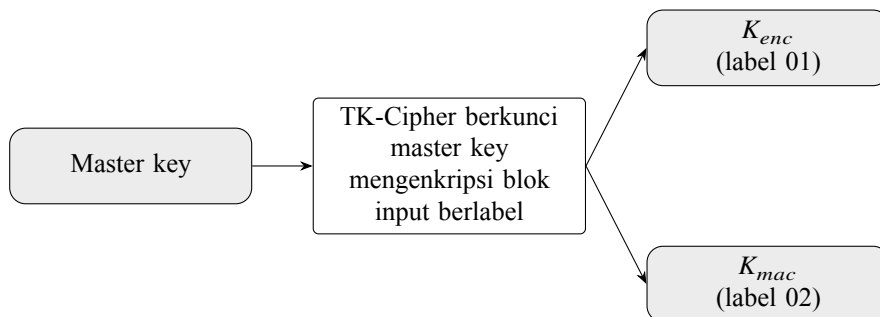
Master key dipakai untuk menurunkan dua key terpisah, satu untuk enkripsi (K_{enc}) dan satu untuk MAC (K_{mac}). KDF dibangun dari TK-Cipher sendiri sebagai fungsi pseudo-acak berbentuk counter, dengan E_M adalah TK-Cipher berkunci master key M (Gambar 3.10):

$$B(\ell, j) = E_M("TKC--KDF" \parallel \ell \parallel 0^{48} \parallel j),$$

$$K_{enc} = B(01, 0) \parallel B(01, 1) \text{ dipotong sepanjang } M,$$

$$K_{mac} = B(02, 0) \parallel B(02, 1) \text{ dipotong sepanjang } M.$$

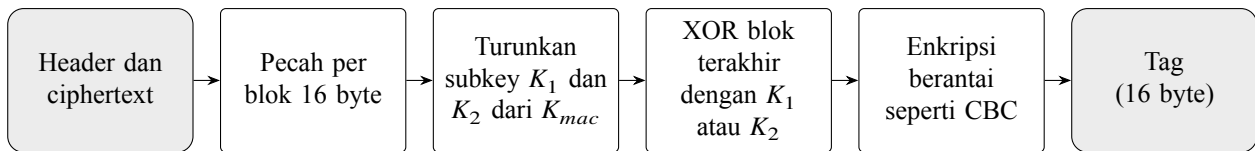
Label ℓ yang berbeda membuat kedua key terpisah. Panjang keluaran sama dengan panjang master key sehingga kekuatan tidak turun untuk key 192 dan 256 bit.



Gambar 3.10: Penurunan key enkripsi dan key MAC

3.9.2 CMAC-TK

MAC yang dipakai adalah CMAC (OMAC1) yang ditulis sendiri dengan $E = \text{TK-Cipher}$ berkunci K_{mac} (Gambar 3.11). Doubling di $GF(2^{128})$ dilakukan dengan menggeser 1 bit ke kiri dan meng-XOR 0×87 jika bit tertinggi bernilai 1. Masukan MAC adalah header dan ciphertext, sehingga mode, IV, dan panjang asli ikut terlindungi. Keluarannya adalah tag 16 byte. Rancangan awal berupa checksum tanpa key tidak dipakai, karena penyerang bisa menghitung ulang checksum setelah memodifikasi ciphertext, sedangkan CMAC berkunci tidak bisa.



Gambar 3.11: Perhitungan tag CMAC-TK

Perbandingan tag memakai fungsi `constant_time_eq` buatan sendiri tanpa `hmac.compare_digest`.

Kode 3.6: Perbandingan tag constant-time (`mac.py`)

```

def constant_time_eq(a, b):
    diff = len(a) ^ len(b)
    for i in range(max(len(a), len(b))):
        x = a[i] if i < len(a) else 0
        y = b[i] if i < len(b) else 0
        diff |= x ^ y
    return diff == 0
  
```

3.10 Format File Ciphertext

Format file bersifat *self-describing*, sehingga dekripsi cukup membutuhkan file dan key. Isinya adalah header 32 byte, ciphertext (kelipatan 16 byte), lalu tag 16 byte (Tabel 3.5).

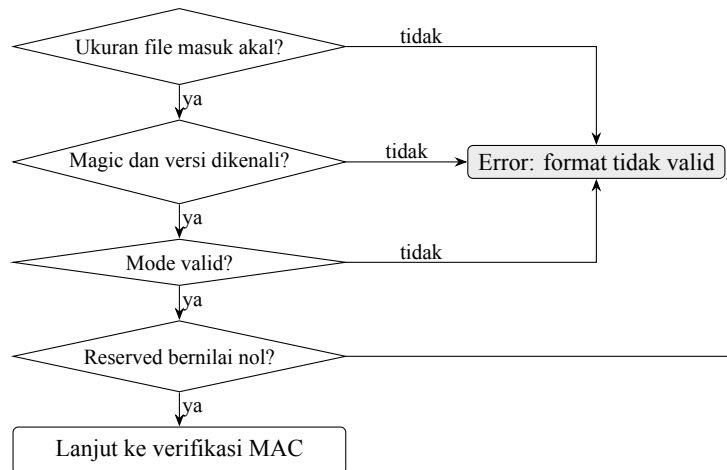
Tabel 3.5: Layout header file

Offset	Ukuran (byte)	Field	Isi
0	4	magic	"TKC1"
4	1	version	0x01
5	1	mode	0 ECB, 1 CBC, 2 CFB, 3 OFB, 4 CTR
6	2	reserved	0x0000
8	16	IV atau counter awal	ECB berisi nol semua
24	8	panjang asli	uint64 big-endian
32	N	ciphertext	N kelipatan 16 dan $N \geq 16$
$32 + N$	16	tag	CMAC atas <code>bytes[0:32+N]</code>

Urutan dekripsi adalah sebagai berikut (Gambar 3.12).

1. Periksa ukuran file, magic, versi, mode yang valid, dan field reserved sama dengan nol. Kegagalan menghasilkan kesalahan format.
2. Turunkan K_{enc} dan K_{mac} , hitung ulang tag, lalu bandingkan secara constant-time. Jika berbeda, proses dihentikan dengan pesan kesalahan autentikasi dan tidak ada keluaran yang ditulis.
3. Dekripsi sesuai mode di header, buang padding, dan cocokkan panjang dengan panjang asli di header.
4. Tulis keluaran ke berkas sementara lalu ganti nama menjadi berkas tujuan, sehingga tidak pernah ada berkas setengah jadi.

Key yang salah otomatis ditolak pada langkah 2, karena K_{mac} yang diturunkan dari key salah berbeda.



Gambar 3.12: Validasi format saat dekripsi

3.11 Rancangan Program

Program berupa CLI dengan tiga perintah. Tabel 3.6 merangkum perintah dan kode keluarnya.

Kode 3.7: Sintaks CLI

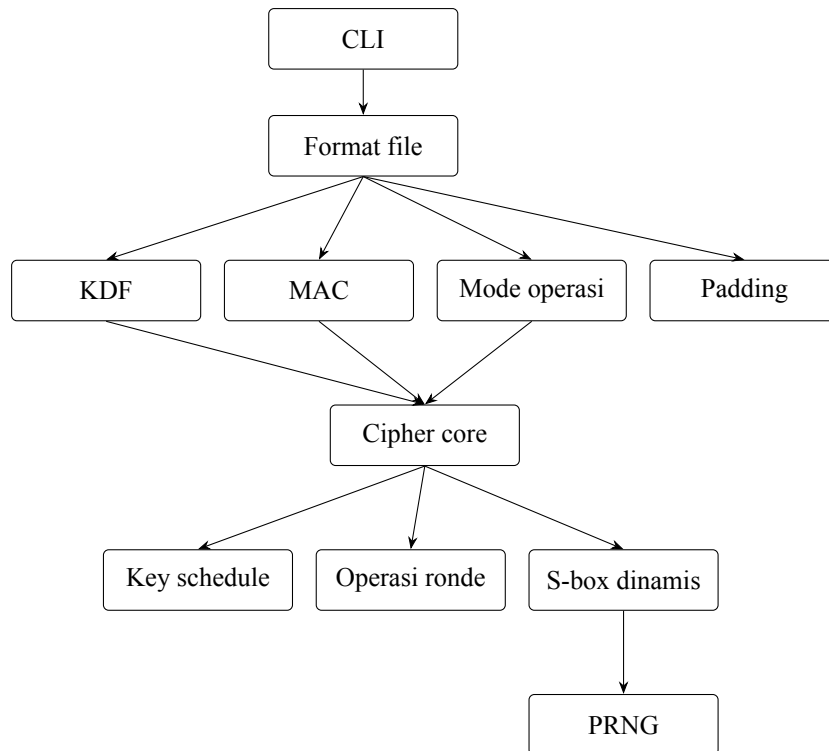
```

tk-cipher enc -i <in> -o <out> (-k <hex> | --key-file <path>) -m {ecb,cbc,cfb,ofb,ctr}
    [--iv <32 hex>]
tk-cipher dec -i <in> -o <out> (-k <hex> | --key-file <path>)
tk-cipher keygen [--bits 128|192|256]
  
```

Tabel 3.6: Kode keluar CLI

Kode	Arti
0	Berhasil
1	Argumen salah, key tidak valid, berkas masukan tidak ada, --iv dipakai di ECB, atau -o sama dengan -i
2	Format berkas ciphertext rusak
3	Autentikasi gagal, yaitu key salah atau berkas telah diubah

Key dimasukkan sebagai hex (32, 48, atau 64 karakter), baik langsung lewat `-k` maupun dari berkas lewat `--key-file` agar key tidak tersimpan di riwayat shell. Program hanya memakai pustaka standar Python pada jalur kriptografi, sehingga berjalan di Linux dan Windows. Pembagian modul ditunjukkan pada Gambar 3.13. Alur enkripsi dan dekripsi lengkap sudah ditunjukkan pada Gambar 3.1 dan 3.2.



Gambar 3.13: Pembagian modul

3.12 Perbedaan dengan AES

TK-Cipher hanya meminjam bentuk state matriks 4x4 dan ukuran blok 128 bit dari AES. Komponen dan matematikanya berbeda (Tabel 3.7).

Tabel 3.7: TK-Cipher dibandingkan AES

Aspek	AES	TK-Cipher
Urutan ronde	SubBytes, ShiftRows, MixColumns, AddRound-Key	AddRoundKey, DiagonalTranspose, Dynamic-Sub, RowRotator, ColumnCascade
S-box	Satu S-box tetap (invers $GF(2^8)$ dan affine)	S-box berbeda per key, Fisher-Yates dengan PRNG ARX dan filter kualitas
Transposisi	ShiftRows, geser byte per baris	Transpose matriks penuh
Pencampuran	MixColumns, perkalian di $GF(2^8)$	Rotasi bit per baris dan penjumlahan mod 256 berantai per kolom
Key schedule	RotWord, SubWord, Rcon	Penyerapan lane key bergantian lewat S-box dinamis dan operasi ronde
Jumlah ronde	10, 12, atau 14	16 untuk semua ukuran key

3.13 Pelajaran dari Rancangan Awal

Rancangan pertama kami tidak merotasi baris 0, sementara transpose memetakan posisi (0, 0) ke dirinya sendiri. Akibatnya byte pertama tiap blok terisolasi sepenuhnya, yaitu $C[0] = f(P[0], K)$, dan tidak pernah bercampur dengan byte lain berapa pun jumlah rondanya. Selain itu, lapisan linear rancangan awal hanya berupa permutasi bit tanpa pencampuran antar byte sehingga diffusion lambat. Rancangan sekarang memperbaiki keduanya, yaitu semua baris dirotasi dan ColumnCascade ditambahkan sebagai pencampur antar byte.

BAB IV

IMPLEMENTASI DAN SIMULASI

4.1 Deskripsi Program

4.1.1 Teknologi dan Dependensi

Program ditulis dalam Python (minimal 3.10, dikembangkan pada 3.14). Seluruh jalur kriptografi hanya memakai pustaka standar yang non-kriptografis. Hasil pemindaian impor pada `src/tk_cipher/` menunjukkan modul berikut saja: `argparse`, `collections`, `dataclasses`, `enum`, `os`, `pathlib`, `struct`, `sys`, dan `tempfile`. Tidak ada `hashlib`, `hmac`, `random`, `secrets`, maupun library kriptografi pihak ketiga. Satu-satunya sumber keacakan dari sistem adalah `os.urandom` untuk IV dan key hasil keygen. Sebuah test otomatis memeriksa impor terlarang ini.

Tabel 4.1: Teknologi yang dipakai

Kebutuhan	Teknologi
Bahasa	Python 3.10 atau lebih baru
Jalur cipher	Pustaka standar saja, tanpa dependency runtime
Antarmuka	<code>argparse</code> (CLI <code>tk-cipher</code>)
Pengelola paket	<code>uv</code> (<code>uv sync</code> , <code>uv run</code>)
Pengujian	<code>pytest</code> dan <code>doctest</code> , lint dengan <code>ruff</code>
Analisis	<code>numpy</code> dan <code>matplotlib</code> , hanya di folder <code>analysis/</code>
Dokumentasi API	<code>pdoc</code> , dihosting di GitHub Pages
Executable	PyInstaller (Linux dan Windows)
CI	GitHub Actions (<code>ruff</code> di Linux, <code>pytest</code> di Linux dan Windows)

4.1.2 Struktur Repositori dan Modul

Tabel 4.2: Modul pada `src/tk_cipher/`

Modul	Isi
<code>prng.py</code>	PRNG TKRand (ARX 4x32 bit)
<code>sbox.py</code>	Pembangkit S-box, perhitungan DU dan NL, tabel inverse
<code>rounds.py</code>	Lima operasi ronde dan inverse-nya
<code>key_schedule.py</code>	Penurunan 17 round key
<code>cipher.py</code>	Kelas TKCipher: <code>encrypt_block</code> , <code>decrypt_block</code>
<code>modes.py</code>	ECB, CBC, CFB, OFB, CTR
<code>padding.py</code>	<code>pad</code> dan <code>unpad</code> PKCS#7
<code>kdf.py</code>	<code>derive_keys</code> , penurunan K_{enc} dan K_{mac}
<code>mac.py</code>	CMAC-TK dan <code>constant_time_eq</code>
<code>fileformat.py</code>	Header, <code>encrypt_file</code> , <code>decrypt_file</code> , <code>verify-then-decrypt</code>
<code>errors.py</code>	Hirarki kesalahan (key, format, autentikasi, padding)
<code>cli.py</code>	Perintah <code>enc</code> , <code>dec</code> , <code>keygen</code> dan kode keluar

Kode 4.1: Struktur repositori

```
tk_cipher
|-- analysis
|   |-- avalanche.py
|   |-- benchmark.py
|   |-- entropy.py
|   |-- histogram.py
|   |-- round_diffusion.py
|   |-- run_all.py
|   `-- sbox_stats.py
|-- dist
|   |-- tk-cipher-linux
|   `-- tk-cipher-windows.exe
|-- docs
|   |-- api
|   |-- diagrams
|   `-- DESIGN.md
|-- src
|   `-- tk_cipher
|-- tests (unit, integration, e2e, data, results)
|-- tools
|   `-- build_api_docs.py
|-- pyproject.toml
|-- README.md
`-- tk-cipher.spec
```

4.1.3 Cara Menjalankan

Kode 4.2: Menjalankan dari source dan executable

```
uv sync
uv run tk-cipher keygen > key.txt
uv run tk-cipher enc -i tests/data/text_small.txt -o out.cbc -m cbc --key-file key.txt
uv run tk-cipher dec -i out.cbc -o back.txt --key-file key.txt

# executable tanpa Python
./dist/tk-cipher-linux enc -i tests/data/text_small.txt -o out.enc -m cbc --key-file key.
txt
.\dist\tk-cipher-windows.exe enc -i tests\data\text_small.txt -o out.enc -m cbc --key-
file key.txt
```

Mode dan IV tersimpan di header, sehingga dec cukup membutuhkan file dan key. Saat IV dibangkitkan otomatis, nilainya dicetak ke *stderr*. Executable dibangun dengan PyInstaller dari `tk-cipher.spec`. Tiap OS harus dibangun di OS masing-masing karena PyInstaller tidak mendukung cross compile. Hasilnya adalah `dist/tk-cipher-linux` dan `dist/tk-cipher-windows.exe`, dan keduanya dilampirkan pada GitHub Release.

4.1.4 Dokumentasi API

Dokumentasi API dibangkitkan dari docstring format Google dan type hint pada `src/tk_cipher/` memakai `pdoc`. Setiap fungsi, kelas, dan konstanta publik memuat deskripsi, parameter beserta tipe datanya, nilai kembali, kesalahan yang mungkin muncul, contoh pemakaian, dan catatan. Contoh pada docstring berupa doctest yang ikut dijalankan `pytest`, sehingga contoh selalu sesuai kode. Hasil HTML disimpan di `docs/api/` dan dihosting publik melalui GitHub Pages dari folder `docs/` pada branch `main`. Tautannya ada di Lampiran A dan README. Untuk membangun ulang, jalankan `uv run python tools/build_api_docs.py`.

4.2 Hasil Enkripsi dan Dekripsi

Simulasi dijalankan pada tiga berkas uji: teks (`text_small.txt`), gambar bitmap (`image.bmp`), dan gambar PNG (`binary.png`) sebagai data biner. Setiap berkas dienkripsi di kelima mode dengan key hex 000102030405060708090a0b0c0d0e0f, didekripsi, lalu dibandingkan dengan aslinya memakai `cmp`. Semua perbandingan menyatakan isi identik byte demi byte (Tabel 4.3).

Tabel 4.3: Hasil enkripsi dan dekripsi di lima mode

Berkas	Mode	Ukuran asli (byte)	Ukuran enkripsi (byte)	Hasil
text_small.txt (teks)	ECB	20.383	20.432	identik
	CBC	20.383	20.432	identik
	CFB	20.383	20.432	identik
	OFB	20.383	20.432	identik
	CTR	20.383	20.432	identik
image.bmp (biner)	ECB	196.662	196.720	identik
	CBC	196.662	196.720	identik
	CFB	196.662	196.720	identik
	OFB	196.662	196.720	identik
	CTR	196.662	196.720	identik
binary.png (biner)	ECB	97.473	97.536	identik
	CBC	97.473	97.536	identik
	CFB	97.473	97.536	identik
	OFB	97.473	97.536	identik
	CTR	97.473	97.536	identik

Perintah yang dipakai untuk kelima mode ditunjukkan pada Kode 4.3. Mode ECB tidak memakai IV, CBC dan OFB memakai IV acak otomatis, CFB memakai IV manual, dan CTR memakai counter awal manual. Mode dan IV tersimpan di header, sehingga perintah dekripsi sama untuk semua mode dan hanya membutuhkan berkas dan key.

Kode 4.3: Enkripsi dan dekripsi untuk kelima mode

```
KEY=000102030405060708090a0b0c0d0e0f  
D=tests/data  
  
tk-cipher enc -i $D/text_small.txt -o t.ecb -m ecb -k $KEY  
tk-cipher enc -i $D/text_small.txt -o t.cbc -m cbc -k $KEY  
tk-cipher enc -i $D/image.bmp -o i.cfb -m cfb -k $KEY --iv 000102030405060708090  
    a0b0c0d0e0f  
tk-cipher enc -i $D/binary.png -o b.ofb -m ofb -k $KEY  
tk-cipher enc -i $D/binary.png -o b.ctr -m ctr -k $KEY --iv  
    00000000000000000000000000000001  
tk-cipher dec -i t.ecb -o t_ecb.txt -k $KEY
```

Selisih ukuran selalu sama dengan overhead format, yaitu header 32 byte, tag 16 byte, dan padding. Contoh perintah dan keluaran untuk satu berkas pada mode CBC:

Kode 4.4: Contoh enkripsi dan dekripsi mode CBC

```
$ tk-cipher enc -i tests/data/text_small.txt -o t.cbc -m cbc -k 000102030405060708090a0b0c0d0e0f
mode: cbc
iv: 09a8776b4747736f187d879897960a40
$ tk-cipher dec -i t.cbc -o t.out -k 000102030405060708090a0b0c0d0e0f
$ cmp tests/data/text_small.txt t.out && echo identik
identik
```

Pada mode CTR, opsi `--iv` dipakai sebagai counter awal. Dengan `--iv` bernilai `00000000000000000000000000000001`, 16 byte IV pada header (offset 8) bernilai persis `00000000000000000000000000000001`, sehingga counter awal ikut tersimpan.

Selain simulasi tiga berkas di atas, skenario uji *end-to-end* dijalankan melalui CLI pada Python dan pada executable Linux. Masing-masing berisi 67 skenario dan seluruhnya lulus (berkas hasil di `tests/results/`, yaitu `e2e_linux_python.md` dan `e2e_linux_tk-cipher-linux.md`). Skenario ini mencakup round-trip delapan berkas (teks, gambar bitmap, PNG, biner sedang 1 MiB, biner besar 5 MiB, berkas berulang, berkas kosong, dan berkas 17 byte) di mode ECB, CBC, CFB, OFB, dan CTR (CBC dan CTR juga dengan `--iv`), pembacaan key dari berkas, serta kasus-kasus galat pada Subbab 4.4.

4.3 Bukti Penanganan Padding

Berkas acak dengan panjang 0, 1, 15, 16, 17, 31, 32, dan 33 byte dienkripsi di mode CBC lalu didekripsi. Ukuran berkas terenkripsi mengikuti rumus $32 + 16 (\lceil n/16 \rceil + 1) + 16$ dengan 32 byte header dan 16 byte tag. Semua berkas hasil dekripsi identik dengan aslinya (Tabel 4.4).

Tabel 4.4: Ukuran berkas terenkripsi untuk berbagai panjang data

Panjang asli n	Padding p	Data ter-pad	Berkas terenkripsi	Hasil dekripsi
0	16	16	64	identik
1	15	16	64	identik
15	1	16	64	identik
16	16	32	80	identik
17	15	32	80	identik
31	1	32	80	identik
32	16	48	96	identik
33	15	48	96	identik

Berkas kosong tetap menghasilkan satu blok penuh, dan berkas yang pas kelipatan 16 (16 dan 32 byte) bertambah satu blok penuh. Keduanya sesuai mekanisme PKCS#7. Pengujian unit juga memeriksa penolakan data ber-padding yang rusak (byte terakhir nol, byte terakhir lebih dari ukuran blok, byte padding tidak seragam, atau panjang bukan kelipatan blok).

4.4 Bukti Verifikasi Integritas

Empat skenario dijalankan terhadap berkas `t.cbc` yang valid. Pada semua skenario penolakan, berkas keluaran tidak dibuat.

Kode 4.5: Tamper test, key salah, dan format rusak

```
# ubah 1 byte ciphertext (offset 100)
$ printf '\x01' | dd of=t.cbc bs=1 seek=100 conv=notrunc
$ tk-cipher dec -i t.cbc -o rusak.txt -k 000102030405060708090a0b0c0d0e0f
tk-cipher: error: MAC verification failed: wrong key or file has been modified
$ echo $?
3 # rusak.txt tidak dibuat

# key salah
$ tk-cipher dec -i asli.cbc -o salah.txt -k 0f0e0d0c0b0a09080706050403020100
tk-cipher: error: MAC verification failed: wrong key or file has been modified
$ echo $?
3 # salah.txt tidak dibuat

# ubah byte mode di header (CBC menjadi CFB)
tk-cipher: error: MAC verification failed: wrong key or file has been modified
exit 3

# berkas terpotong (40 byte)
tk-cipher: error: Encrypted file is too short
exit 2
```

Tabel 4.5: Ringkasan uji integritas

Skenario	Kode keluar	Berkas keluaran	Alasan
Ubah 1 byte ciphertext	3	tidak dibuat	Tag tidak cocok
Key salah	3	tidak dibuat	K_{mac} berbeda sehingga tag tidak cocok
Ubah byte mode di header	3	tidak dibuat	Header ikut di-MAC
Berkas terpotong	2	tidak dibuat	Ukuran di bawah minimum 64 byte

Perubahan header terdeteksi sebagai kegagalan autentikasi, bukan sebagai kesalahan format, karena header yang secara struktur masih valid (mode 2 adalah CFB) baru ditolak saat tag diperiksa. Pesan kesalahan untuk key salah dan berkas diubah sengaja sama agar tidak membocorkan informasi tambahan. Unit test juga menguji perubahan 1 byte pada setiap daerah berkas (header, IV, panjang, ciphertext, dan tag).

4.5 Test Vector dan Jejak Satu Blok

Tabel 4.6 memuat test vector internal untuk ketiga ukuran key. Nilainya dihasilkan implementasi, disimpan sebagai fixture, dan diperiksa oleh unit test. Plaintext yang sama dipakai pada semuanya, dan tiap pasangan dapat didekripsi kembali menjadi identik.

Tabel 4.6: Test vector TK-Cipher (plaintext 00112233445566778899aabbccddeeff)

Key	Nilai key (hex)	Ciphertext	DU	NL	Percobaan
128 bit	000102030405060708090a0b0c0d0e0f	9dc8f80581c878ae5f95b6ffc2d3b86b	12	94	1
192 bit	000102030405060708090a0b0c0d0e0f1011121314151617	ea4f7ee0b96f039fb5a74f5bb2b64afa	12	94	1
256 bit	000102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f	fae80ec4357c9a7c027afd53f3747b32	12	96	1

Sebagai ilustrasi, Tabel 4.7 menunjukkan jejak satu blok masukan cipher melewati ronde pertama. Contohnya memakai key enkripsi turunan dari master key 000102...0f dan blok masukan 78747e7f7b1e1d1c

1312111017161514. Blok itu adalah hasil XOR antara plaintext `hello` yang sudah dipad dengan IV `101112...1f` pada mode CBC.

Tabel 4.7: Keadaan state selama ronde pertama

Langkah	State (hex)
Masukan	78747e7f7b1e1d1c1312111017161514
AddRoundKey (RK_0)	f1366ced0c3f9143a7498c7e1ffc4cae
DiagonalTranspose	f10ca71f363f49fc6c918c4ced437eae
DynamicSub	4a86a45eb3986d02459c6c1c28c8df35
RowRotator	950d48bc9cc36815b38d8388646f9a94
ColumnCascade	ddd915a931d0b0d1e45d335948cccded
Sesudah ronde 16 dan whitening (RK_{16})	280164a53d1662c50f0eea8bf7bc7b2e

Contoh header berkas hasil enkripsi (48 byte pertama dari `t.cbc`) memperlihatkan format pada Tabel 3.5:

Kode 4.6: Header berkas terenkripsi

```
00000000: 544b 4331 0101 0000 09a8 776b 4747 736f TKC1.....wkGGso
00000010: 187d 8798 9796 0a40 0000 0000 0000 4f9f .}.....@.....O.
00000020: d7bb e5cb 1565 df5d 54b8 25ec 0f91 d978 .....e.]T.%....x
```

Byte 0 sampai 3 adalah magic `TKC1`, byte 4 adalah versi 1, byte 5 adalah mode 1 (CBC), byte 6 dan 7 adalah reserved nol, byte 8 sampai 23 adalah IV, dan byte 24 sampai 31 adalah panjang asli plaintext (`0x4f9f` atau 20.383 byte). Ciphertext dimulai pada offset 32.

4.6 Pengujian Otomatis

Seluruh komponen diuji dengan `pytest`. Dari 437 test yang terkumpul, 367 test *non-slow* dijalankan pada pengembangan dan CI, sedangkan 70 test *slow* (berkas besar) dijalankan secara lokal. Seluruh 367 test *non-slow* lulus pada pengujian terakhir (waktu sekitar 40 detik). Terdapat juga sekitar 49 doctest dari contoh di docstring yang ikut tercakup. Tabel 4.8 merangkum cakupan berkas test.

Tabel 4.8: Cakupan berkas pengujian

Berkas	Jumlah	Yang diuji
<code>test_rounds.py</code>	20	Inverse tiap operasi ronde
<code>test_sbox.py</code>	35	Permutasi valid, DU, NL, fixed point, determinisme per key
<code>test_key_schedule.py</code>	21	17 round key, key 128, 192, 256 bit
<code>test_cipher.py</code>	15	Round-trip blok dan test vector
<code>test_modes.py</code>	70	Kelima mode, round-trip, panjang dan IV
<code>test_padding.py</code>	26	Edge case padding dan unpad tidak valid
<code>test_kdf.py</code> , <code>test_mac.py</code>	7 dan 11	Pemisahan key, tag CMAC, constant-time compare
<code>test_fileformat.py</code>	50	Header, verify-then-decrypt, tamper tiap daerah
<code>test_integration.py</code>	97	Round-trip semua mode dan ukuran key, CLI subprocess, impor terlarang
<code>test_cli.py</code> , <code>test_smoke.py</code>	30 dan 6	Argumen, kode keluar, smoke test

Integrasi berkelanjutan (GitHub Actions) menjalankan lint `ruff` di Linux, serta `pytest -m "not slow"` di Linux dan Windows dengan Python 3.10 dan 3.14, pada setiap push ke `main` dan setiap pull request. Karena seluruh anggota tim memakai Linux, skenario *end-to-end* dijalankan di Linux, sedangkan kesesuaian di Windows dicakup oleh CI tersebut.

BAB V

ANALISIS KEAMANAN

Seluruh pengujian dijalankan pada kelima mode operasi. Skrip berada di `analysis/` dan hasil mentahnya (CSV, ringkasan, dan gambar) tersimpan di `tests/results/`. Semua analisis dapat dibuat ulang dengan `uv run python analysis/run_all.py`. Seed acak dicatat pada tiap pengujian agar hasil dapat direproduksi.

5.1 Avalanche Effect

5.1.1 Flip 1 Bit Plaintext

Pada tiap mode diambil 1000 sampel dengan key 128 bit acak dan plaintext sepanjang 4 blok. Satu bit plaintext dibalik secara acak, lalu dihitung persentase bit ciphertext yang berubah. IV dibuat sama untuk dua proses enkripsi, karena IV acak akan membuat hasil otomatis mendekati 50% dan menyesatkan. Tabel 5.1 merangkum hasilnya. Nilai “blok diubah” adalah blok ciphertext yang posisinya sama dengan blok plaintext yang dibalik.

Tabel 5.1: Avalanche flip 1 bit plaintext (persen bit ciphertext berubah, 1000 sampel per mode)

Mode	Blok yang diubah				Blok sesudahnya	Total 4 blok
	Mean	Min	Max	Std		
ECB	50,01	35,94	66,41	4,46	0,00	12,50
CBC	49,80	34,38	64,84	4,38	50,04	30,93
CFB	0,78	0,78	0,78	0,00	50,04	18,96
OFB	0,78	0,78	0,78	0,00	0,00	0,20
CTR	0,78	0,78	0,78	0,00	0,00	0,20

Hasilnya sesuai ekspektasi tiap mode dan perlu dibaca per mode.

- ECB. Hanya blok yang dibalik yang berubah, dengan rata-rata 50,01%. Blok lain tetap sama, sehingga rata-rata total 4 blok hanya 12,50%. Angka rendah ini adalah sifat mode, bukan kelemahan cipher.
- CBC. Blok yang dibalik dan seluruh blok sesudahnya berubah sekitar 50% karena ciphertext sebelumnya diumpankan ke enkripsi berikutnya.
- CFB. Pada blok yang dibalik hanya tepat 1 bit yang berubah (0,78% dari 128 bit), karena $C_i = P_i \oplus E(\cdot)$. Blok sesudahnya berubah sekitar 50% karena C_i yang berubah masuk ke E .
- OFB dan CTR. Keystream tidak bergantung pada plaintext, sehingga tepat 1 bit ciphertext yang berubah. Ini sifat mode stream dan sesuai teori.

Pada blok yang terdampak (blok yang diubah pada ECB dan CBC, serta blok sesudahnya pada CBC dan CFB), rata-rata konsisten sekitar 50% dengan simpangan baku sekitar 4,4. Ini menunjukkan diffusion cipher bekerja baik di seluruh posisi bit yang dibalik.

5.1.2 Flip 1 Bit Key

Master key dibalik 1 bit dengan plaintext dan IV tetap. Ukuran key bergantian 128, 192, dan 256 bit. Dua jalur diukur: jalur *cipher* (TKCipher ditambah mode, 300 sampel per mode) dan jalur *full* (encrypt_bytes lengkap dengan KDF dan MAC, hanya bagian ciphertext yang dibandingkan, 50 sampel per mode). Tabel 5.2 memperlihatkan hasilnya.

Tabel 5.2: Avalanche flip 1 bit key (persen bit ciphertext berubah)

Jalur	Mode	Mean	Min	Max	Std
Cipher	ECB	49,98	43,95	56,25	2,18
	CBC	50,36	43,36	56,45	2,38
	CFB	49,91	43,55	55,86	2,27
	OFB	49,87	41,41	55,86	2,35
	CTR	49,85	43,55	54,88	2,11
Full	ECB	50,44	46,41	55,16	1,80
	CBC	49,63	44,84	53,75	2,05
	CFB	49,67	45,47	52,03	1,56
	OFB	50,18	46,88	53,91	1,68
	CTR	50,30	46,09	55,78	1,92

Perubahan 1 bit key mengubah sekitar 50% bit ciphertext di semua mode dan di kedua jalur, dengan rata-rata antara 49,63% dan 50,44%. Hasil ini menunjukkan bahwa confusion antara key dan ciphertext baik. Pada flip key, S-box dan seluruh round key dibangun ulang dari key yang berbeda, sehingga semua komponen yang bergantung key ikut berubah.

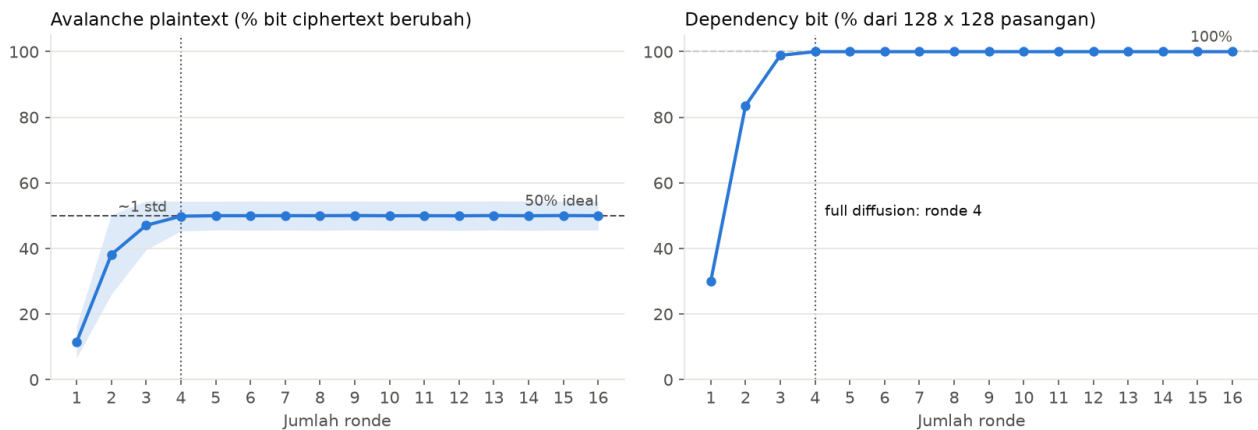
5.2 Round Diffusion dan Justifikasi 16 Ronde

Untuk menentukan jumlah ronde, $\text{TKCipher}(\text{key}, \text{rounds}=r)$ diukur untuk $r = 1, \dots, 16$. Diambil 8 key acak dan 8 plaintext acak per key, lalu seluruh 128 bit plaintext dibalik satu per satu, sehingga terdapat 8192 sampel per ronde dan 64 sampel per bit masukan. Selain rata-rata avalanche, dihitung juga *dependency*, yaitu jumlah pasangan (bit masukan, bit keluaran) yang pernah berubah dari maksimum $128 \times 128 = 16384$. *Full diffusion* dianggap tercapai jika seluruh 16384 pasangan teramati dan rata-rata avalanche berada dalam 1 poin persen dari 50%.

Tabel 5.3: Avalanche dan dependency per jumlah ronde

Ronde	Avalanche mean (%)	Min (%)	Max (%)	Std	Dependency (%)
1	11,44	1,56	29,69	4,83	30,02
2	38,12	4,69	67,19	12,10	83,53
3	47,04	11,72	65,62	7,43	98,88
4	49,83	26,56	65,62	4,52	100,00
5	49,99	32,81	65,62	4,36	100,00
6	49,98	32,03	67,19	4,44	100,00
8	49,98	31,25	65,62	4,46	100,00
10	49,96	35,16	67,97	4,41	100,00
12	49,95	32,81	66,41	4,45	100,00
14	49,96	33,59	66,41	4,45	100,00
16	49,97	32,81	66,41	4,41	100,00

TK-Cipher round diffusion · 8192 sampel per ronde · seed 20261005



Gambar 5.1: Avalanche dan dependency bit per jumlah ronde

Full diffusion pertama kali tercapai pada ronde ke-4. Pada ronde ke-3 dependency sudah 98,88% dan avalanche 47,04%, sehingga ronde ke-4 adalah batas yang konservatif. Setelah itu avalanche stabil di sekitar 50% sampai ronde ke-16. Dengan 16 ronde, margin keamanan adalah 12 ronde tambahan atau empat kali ronde full diffusion. Pada AES-128 margin serupa adalah 10 ronde terhadap full diffusion pada ronde ke-2 (lima kali). Tabel di atas menampilkan sebagian ronde, dan data lengkap 16 ronde ada pada `tests/results/round_diffusion.md`.

5.3 Entropi dan Uji Chi-square

Setiap berkas pada `tests/data/` dienkripsi dengan `encrypt_bytes` di kelima mode, key dan IV tetap, lalu dihitung entropi Shannon dan statistik chi-square atas bagian ciphertext saja (tanpa header 32 byte dan tag 16 byte). Lolos berarti $\chi^2 \leq 293,248$ ($\alpha = 0,05$, derajat kebebasan 255). Kolom terakhir adalah jumlah blok ciphertext 16 byte yang berulang.

Tabel 5.4: Entropi (bit per byte) dan chi-square ciphertext

Berkas	Mode	H plain	H cipher	χ^2	Lolos	Blok ulang
binary.png	ECB	7,9974	7,9981	260,3	ya	0
	CBC	7,9974	7,9981	256,3	ya	0
	CFB	7,9974	7,9983	233,7	ya	0
	OFB	7,9974	7,9982	248,4	ya	0
	CTR	7,9974	7,9982	238,4	ya	0
image.bmp	ECB	1,9640	6,1764	1.058.193,2	tidak	12.240
	CBC	1,9640	7,9990	268,8	ya	0
	CFB	1,9640	7,9989	305,4	tidak	0
	OFB	1,9640	7,9990	263,0	ya	0
	CTR	1,9640	7,9991	256,2	ya	0
medium.bin	ECB	7,9999	7,9998	225,3	ya	0
	CBC	7,9999	7,9998	238,3	ya	0
	CFB	7,9999	7,9998	249,7	ya	0
	OFB	7,9999	7,9998	251,6	ya	0
	CTR	7,9999	7,9998	229,9	ya	0
repetitive.bin	ECB	4,0000	3,8781	1.113.840,1	tidak	4.095
	CBC	4,0000	7,9972	254,0	ya	0
	CFB	4,0000	7,9971	266,2	ya	0
	OFB	4,0000	7,9977	213,9	ya	0
	CTR	4,0000	7,9975	224,8	ya	0
text_small.txt	ECB	4,3551	7,9849	421,0	tidak	385
	CBC	4,3551	7,9891	304,7	tidak	0
	CFB	4,3551	7,9916	238,2	ya	0
	OFB	4,3551	7,9913	245,6	ya	0
	CTR	4,3551	7,9917	234,1	ya	0

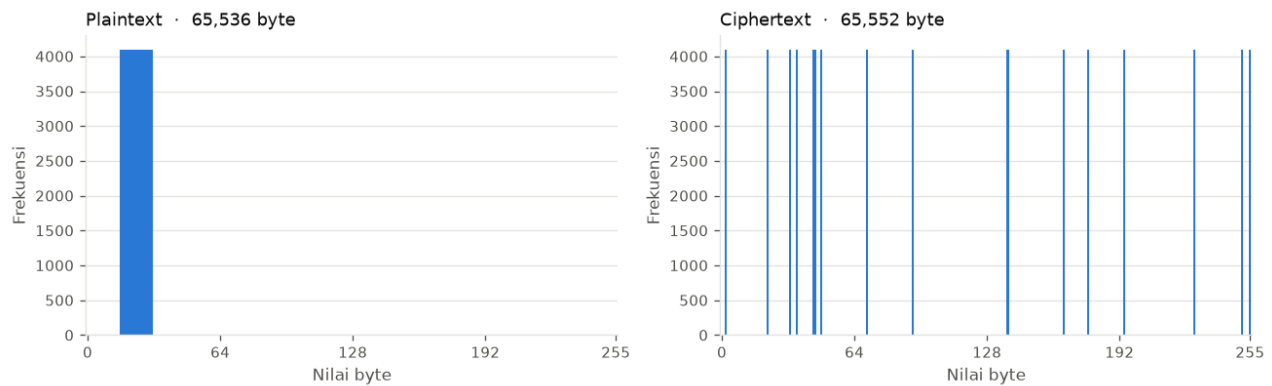
Hasilnya sebagai berikut.

- CBC, CFB, OFB, dan CTR menghasilkan entropi ciphertext paling rendah 7,9891 bit per byte pada semua berkas, termasuk teks dan gambar yang entropi plaintext-nya jauh di bawah 8 (1,9640 untuk gambar dan 4,3551 untuk teks). Sebanyak 18 dari 20 kombinasi lolos chi-square. Dua yang tidak lolos, yaitu image.bmp di CFB ($\chi^2 = 305,4$) dan text_small.txt di CBC ($\chi^2 = 304,7$), hanya sedikit di atas batas 293,2 dan tanpa blok berulang. Pada $\alpha = 0,05$, data yang benar-benar acak diperkirakan gagal sekitar 5% (sekitar 1 dari 20), sehingga ini kami baca sebagai fluktuasi statistik, bukan pola.
- ECB gagal total pada berkas yang punya blok kembar. Pada repetitive.bin, entropi ciphertext hanya 3,8781 bit per byte dan $\chi^2 = 1.113.840$. Berkas ini berisi pola 16 byte yang sama berulang, dan 4095 dari 4097 blok ciphertext adalah duplikat. Kebocoran yang sama terlihat pada image.bmp (12.240 blok berulang, $H = 6,1764$) dan text_small.txt (385 blok berulang).
- ECB pada binary.png dan medium.bin lolos karena plaintext-nya tidak memiliki blok 16 byte yang kembar, bukan karena ECB aman.

5.4 Histogram Frekuensi Byte

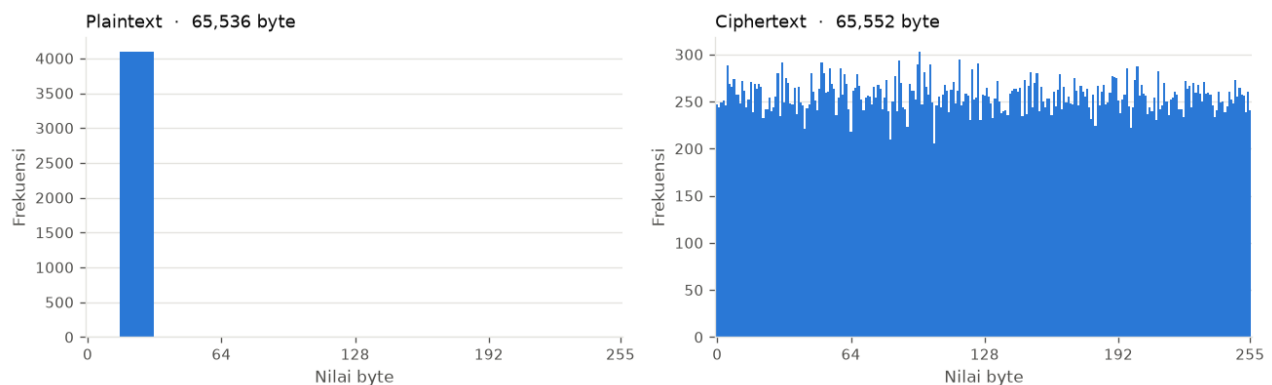
Histogram membandingkan frekuensi tiap nilai byte pada plaintext dan ciphertext. Pada Gambar 5.2 dan 5.3, plaintext berulang hanya memakai satu rentang sempit nilai byte. Pada ECB, ciphertext-nya hanya memakai 16 nilai byte dengan frekuensi sangat tinggi, yang menandakan blok kembar masih terlihat. Pada CBC, histogram ciphertext rata di seluruh 256 nilai.

Histogram byte repetitive.bin · mode ECB · seed 20261006



Gambar 5.2: Histogram repetitive.bin mode ECB

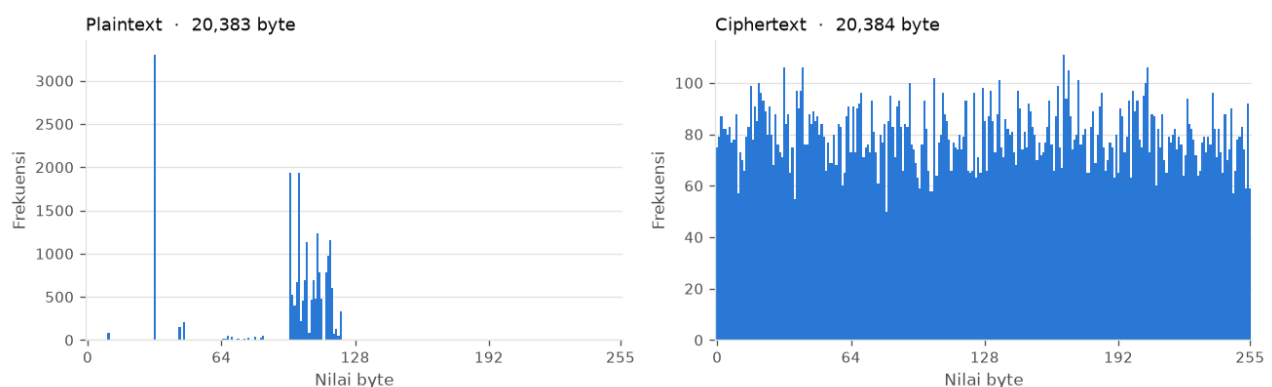
Histogram byte repetitive.bin · mode CBC · seed 20261006



Gambar 5.3: Histogram repetitive.bin mode CBC

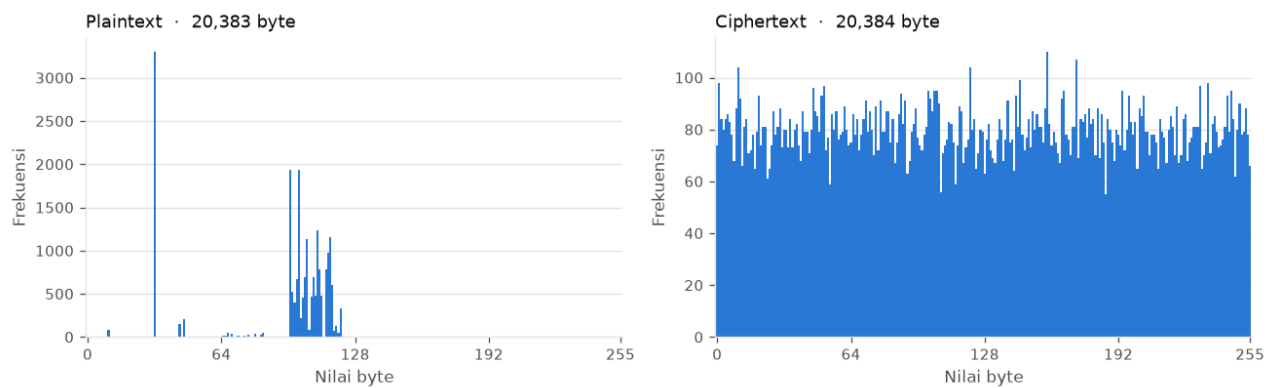
Pada teks (Gambar 5.4 dan 5.5), plaintext hanya memakai nilai ASCII tertentu dengan frekuensi yang sangat timpang (misalnya spasi mencapai sekitar 3300 kemunculan). Pada ECB dan CBC, ciphertext sama-sama tampak hampir rata dengan frekuensi sekitar 50 sampai 110 per nilai byte. Histogram frekuensi byte tunggal memang tidak dapat memperlihatkan blok 16 byte yang berulang, sehingga kebocoran ECB pada teks baru terlihat dari 385 blok berulang dan nilai $\chi^2 = 421,0$ pada Tabel 5.4, sedangkan CBC tanpa blok berulang.

Histogram byte text_small.txt · mode ECB · seed 20261006



Gambar 5.4: Histogram text_small.txt mode ECB

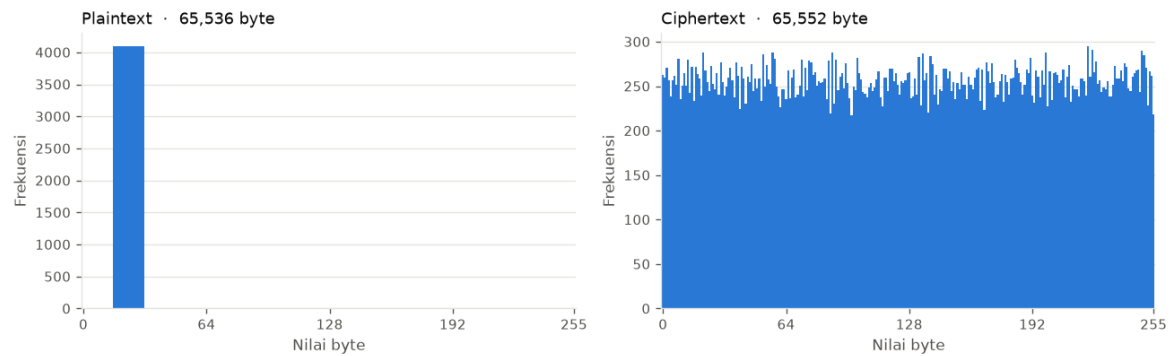
Histogram byte text_small.txt · mode CBC · seed 20261006



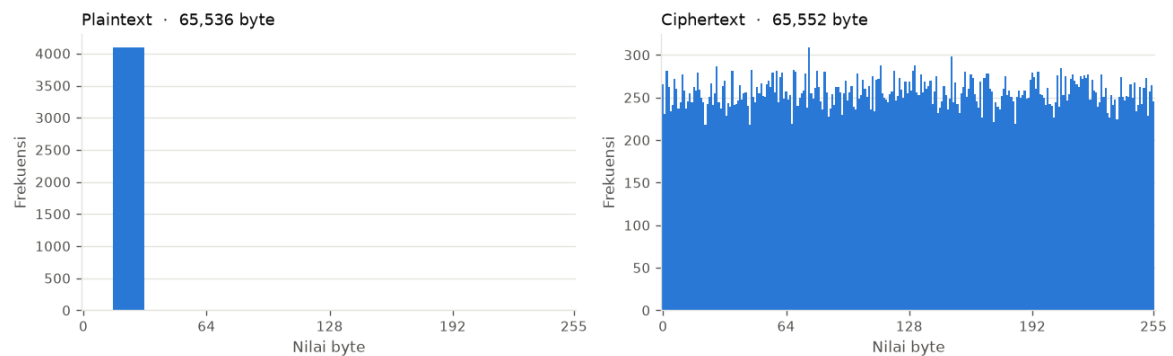
Gambar 5.5: Histogram text_small.txt mode CBC

Agar kelima mode terwakili, Gambar 5.6 menampilkan histogram `repetitive.bin`, input yang paling menantang, pada mode CFB, OFB, dan CTR. Ketiganya rata di seluruh 256 nilai byte seperti CBC, tanpa pola yang tersisa dari plaintext. Hal ini sesuai dengan entropi 7,9971 sampai 7,9977 bit per byte pada Tabel 5.4.

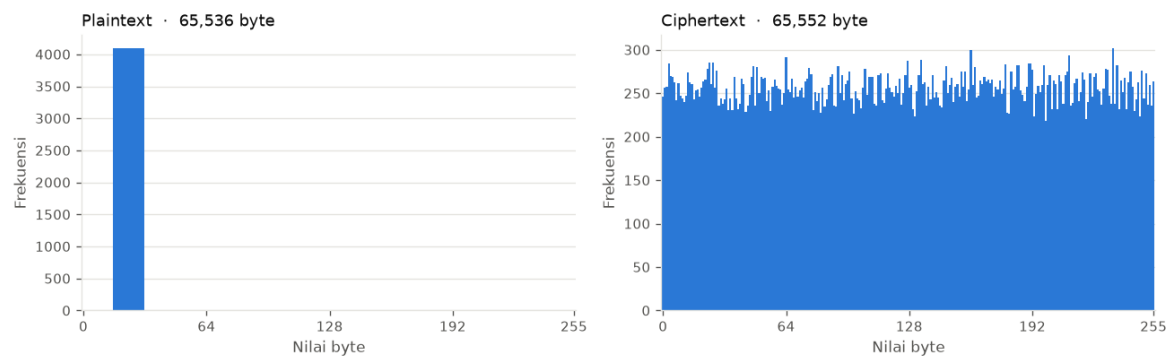
Histogram byte repetitive.bin · mode CFB · seed 20261006



Histogram byte repetitive.bin · mode OFB · seed 20261006



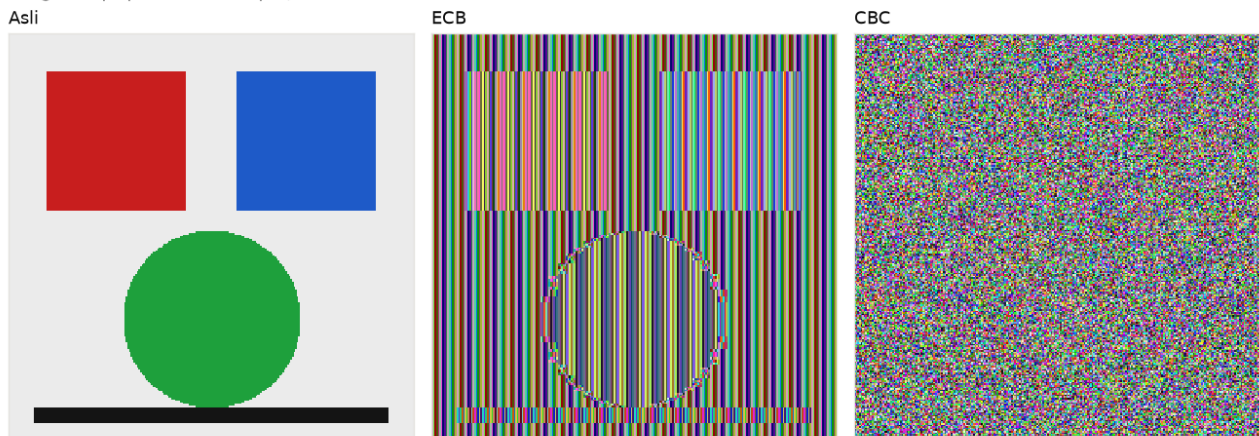
Histogram byte repetitive.bin · mode CTR · seed 20261006



Gambar 5.6: Histogram repetitive.bin pada mode CFB, OFB, dan CTR

Perbedaan ECB dan CBC paling jelas terlihat pada gambar. Gambar 5.7 menampilkan piksel `image.bmp` yang dienkripsi dengan header BMP tetap utuh. Pada ECB, bentuk gambar (dua persegi dan lingkaran) masih dapat dikenali. Pada CBC, gambar menjadi derau acak. Histogram lengkap untuk kelima berkas di kelima mode (25 gambar) ada pada `tests/results/`.

image.bmp: piksel dienkrpsi, header BMP utuh · seed 20261006



Gambar 5.7: Gambar asli, hasil ECB, dan hasil CBC

Kebocoran ECB juga dapat ditunjukkan langsung pada tiga blok plaintext yang identik (48 byte huruf A) dengan key yang sama. Berikut ciphertext tiga blok pertama (tanpa header):

Kode 5.1: Blok identik pada ECB dan CBC

```
ECB:
59d69afa58224d9c1f0bafaa17340c54
59d69afa58224d9c1f0bafaa17340c54
59d69afa58224d9c1f0bafaa17340c54
CBC:
f11a4a10de1b6a7b27736ca8d0b30b16
664402f3c7dfa8d7624d36568d68443c
f2009781b730c06945047d3ba3bb87ff
```

Pada ECB ketiga ciphertext sama persis, sedangkan pada CBC ketiganya berbeda.

5.5 Statistik S-box

Untuk menilai kualitas S-box dinamis, 100 key acak per ukuran (128, 192, dan 256 bit, total 300 key) dilewatkan ke `generate_sbox`. Dicatat DU, NL, jumlah percobaan, dan waktu pembangkitan. Selain itu, 600 kandidat pertama (2 per key) diukur sebelum filter untuk meninjau ambang.

Tabel 5.5: Statistik S-box sesudah filter (300 key)

Statistik	Min	Maks	Rata-rata	Median
DU	10	12	11,11	12
NL	90	98	93,14	94
Percobaan	1	3	1,18	1
Waktu (ms)	26,75	127,35	34,23	28,17

Sebanyak 44,3% S-box memiliki DU 10 dan 55,7% memiliki DU 12. Nilai NL tersebar pada 90 (11,0%), 92 (34,7%), 94 (41,3%), 96 (12,3%), dan 98 (0,7%). Sebanyak 84,3% key lolos pada percobaan pertama, 13,3% pada percobaan kedua, dan 2,3% pada percobaan ketiga. Sebelum filter, 7,8% kandidat memiliki DU di atas 12 dan 5,3% memiliki NL di bawah 90, sehingga filter membuang ekor terlemah tersebut.

Tabel 5.6: Perbandingan ambang filter

Ambang	Lolos (%)	Ekspektasi percobaan	Estimasi waktu
$DU \leq 12, NL \geq 90$ (dipakai)	87,2	1,15	sekitar 54 ms
$DU \leq 14, NL \geq 88$	98,3	1,02	sekitar 47 ms
$DU \leq 12, NL \geq 92$	77,0	1,30	sekitar 61 ms
$DU \leq 10, NL \geq 90$	38,7	2,59	sekitar 121 ms
$DU \leq 10, NL \geq 92$	35,7	2,80	sekitar 131 ms

Ambang $DU \leq 12$ dan $NL \geq 90$ dipertahankan. Memperketat ke $DU \leq 10$ membutuhkan sekitar 2,6 percobaan, atau 2,3 kali lebih lama untuk satu tingkat DU . Permutasi acak 8 bit memang umumnya berada di DU 10 sampai 12. Hal ini jauh dari S-box aljabar seperti AES (DU 4, NL 112). Keamanan TK-Cipher bertumpu pada 16 ronde dengan margin empat kali terhadap full diffusion, bukan pada S-box yang optimal.

5.6 Kinerja: Berkas Kecil, Sedang, dan Besar

Kinerja diukur dengan `encrypt_file` dan `decrypt_file` lengkap termasuk KDF, MAC, dan I/O. Nilai adalah median dari beberapa ulangan (5 ulangan untuk berkas kecil, 3 untuk sedang, 1 untuk besar). Pengukuran dilakukan pada Linux 6.18 dengan AMD Ryzen 7 7730U dan CPython 3.14.7. Seluruh round-trip pada pengukuran ini berhasil.

Tabel 5.7: Throughput enkripsi dan dekripsi

Kategori	Berkas	Ukuran (byte)	Mode	Enc (s)	Dec (s)	Enc (KiB/s)	Dec (KiB/s)
Kecil			ECB	2,275	2,236	41,84	42,57
			CBC	2,302	2,304	41,35	41,32
	binary.png	97.473	CFB	1,076	1,068	88,44	89,14
			OFB	2,321	2,318	41,01	41,07
			CTR	1,069	1,072	89,07	88,77
			ECB	1,597	1,607	40,08	39,82
			CBC	1,608	1,600	39,79	40,01
	repetitive.bin	65.536	CFB	1,610	1,402	39,75	45,65
			OFB	0,742	0,750	86,21	85,33
			CTR	1,607	1,601	39,82	39,97
			ECB	0,288	0,287	69,10	69,26
			CBC	0,292	0,290	68,07	68,59
	text_small.txt	20.383	CFB	0,293	0,292	67,93	68,27
			OFB	0,628	0,632	31,67	31,48
			CTR	0,630	0,630	31,60	31,59
Sedang			ECB	17,993	15,664	56,91	65,37
			CBC	18,850	21,330	54,33	48,01
	medium.bin	1.048.576	CFB	22,530	25,282	45,45	40,50
			OFB	18,365	18,249	55,76	56,11
			CTR	18,945	20,558	54,05	49,81
Besar			ECB	306,299	125,654	16,72	40,75
			CBC	131,195	120,118	39,03	42,62
	large.bin	5.242.880	CFB	130,739	139,547	39,16	36,69
			OFB	138,437	112,142	36,98	45,66
			CTR	124,235	113,580	41,21	45,08

Throughput berada pada kisaran 30 sampai 90 KiB/s untuk berkas kecil dan sedang, dan sekitar 37 sampai 46 KiB/s untuk berkas besar 5 MiB (dengan satu pengecualian pada ECB enkripsi yang hanya satu ulangan). Kin-

erja ini rendah karena implementasi berupa Python murni tanpa optimasi, dan setiap blok melewati 16 ronde yang masing-masing terdiri dari lima operasi pada daftar byte. Perbedaan antar mode tidak sistematis. Semua mode memanggil operasi blok cipher satu kali per blok data, sehingga selisih yang terlihat kemungkinan besar berasal dari beban sistem saat pengukuran dan jumlah ulangan yang kecil, bukan dari kompleksitas mode. Nilai 16,72 KiB/s pada ECB berkas besar berasal dari satu ulangan tunggal dan kemungkinan besar merupakan gangguan beban, karena dekripsi mode yang sama mencapai 40,75 KiB/s.

Selain throughput berkas, waktu *key setup*, yaitu pembuatan S-box dinamis dan 17 round key lewat `TKCipher(key)`, diukur 20 kali per ukuran key (Tabel 5.8). Biaya ini tetap dan kecil dibanding waktu enkripsi berkas.

Tabel 5.8: Waktu key setup

Ukuran key	Sampel	Median (ms)	Min (ms)	Maks (ms)
128 bit	20	27,93	27,57	59,75
192 bit	20	28,47	28,07	61,15
256 bit	20	28,87	28,03	57,26

5.7 Pembahasan dan Keterbatasan

Dari seluruh pengujian, kesimpulan yang dapat ditarik adalah sebagai berikut. Confusion dan diffusion tercapai, yang terlihat dari avalanche sekitar 50% pada plaintext dan key serta full diffusion pada ronde ke-4. Cipher-text pada mode CBC, CFB, OFB, dan CTR menyerupai data acak menurut entropi, chi-square, dan histogram. ECB membocorkan pola pada data berpola, sesuai teori dan perlu dihindari untuk data semacam itu. Integritas berjalan, yaitu tamper dan key salah ditolak sebelum dekripsi.

Namun, hasil ini perlu dibaca dengan batasan berikut.

- Belum ada kriptanalisis formal: pengujian bersifat empiris dan statistik. Tidak ada bukti ketahanan terhadap kriptanalisis diferensial, linear, *related-key*, atau serangan lain. Jumlah ronde dipilih dari pengukuran diffusion dan margin, bukan dari bukti keamanan.
- S-box lebih lemah daripada S-box aljabar: DU 10 sampai 12 dan NL 90 sampai 98 dibandingkan DU 4 dan NL 112 pada AES. S-box dinamis memberi keragaman per key, tetapi itu bukan jaminan ketahanan.
- PRNG TKRand bukan CSPRNG: TKRand hanya dipakai untuk membangun S-box yang deterministik per key. IV dan key acak memakai `os.urandom`.
- Waktu eksekusi tidak konstan: hanya perbandingan tag yang constant-time. Pencarian tabel S-box di Python rentan terhadap serangan waktu atau cache.
- Pemakaian ulang IV atau counter: pada OFB dan CTR dengan key yang sama, pemakaian ulang membocorkan XOR dua plaintext. IV acak otomatis adalah perlindungan bawaan, tetapi opsi `--iv` manual bisa disalahgunakan.
- Kinerja dan memori: implementasi Python murni lambat, dan seluruh berkas dibaca ke memori sekaligus sehingga tidak cocok untuk berkas yang sangat besar.
- Pengujian Windows hanya lewat CI: seluruh anggota tim memakai Linux, sehingga skenario *end-to-end* dijalankan di Linux. Kesesuaian di Windows ditunjukkan oleh `pytest` pada runner `windows-latest` di CI dan oleh executable yang dibangun di sana, tanpa hasil E2E manual di Windows.
- Input key: key dimasukkan sebagai hex, bukan kata sandi, sehingga tidak ada penurunan key dari password.

BAB VI

KESIMPULAN DAN SARAN

6.1 Kesimpulan

TK-Cipher berhasil dirancang dan diimplementasikan sebagai block cipher SPN 128 bit dengan key 128, 192, atau 256 bit, 16 ronde, dan whitening akhir. Setiap ronde terdiri dari AddRoundKey, DiagonalTranspose, DynamicSub, RowRotator, dan ColumnCascade. S-box dibangkitkan dari key lewat PRNG ARX buatan sendiri dengan filter differential uniformity dan nonlinearity, dan 17 round key diturunkan dari master key lewat key schedule sendiri. Seluruh komponen kriptografi, yaitu cipher, lima mode operasi, padding PKCS#7, KDF, dan CMAC, ditulis dari nol tanpa library kriptografi pihak ketiga.

Program enkripsi file mendukung teks dan biner di kelima mode, memakai Encrypt-then-MAC dengan key enkripsi dan key MAC yang terpisah, memverifikasi tag secara constant-time sebelum dekripsi, serta memiliki format file yang mendeskripsikan dirinya sendiri. Round-trip berhasil identik pada seluruh berkas uji, edge case padding (0, 1, 15, 16, 17, 31, 32, dan 33 byte) tertangani, dan berkas yang diubah atau didekripsi dengan key salah ditolak tanpa menghasilkan plaintext. Program berjalan di Linux dan Windows, tersedia sebagai executable, dan fungsi publiknya didokumentasikan lewat API docs yang dihosting.

Hasil analisis menunjukkan perubahan 1 bit plaintext dan key mengubah sekitar 50% bit ciphertext pada blok yang terdampak di semua mode (dengan perilaku khas tiap mode), diffusion penuh tercapai pada ronde ke-4 sehingga 16 ronde memberi margin empat kali lipat, dan ciphertext CBC, CFB, OFB, dan CTR memiliki entropi sekitar 7,99 bit per byte dengan histogram yang rata. ECB membocorkan pola pada data berulang seperti yang diprediksi teori. Di sisi lain, S-box dinamis memiliki kualitas di bawah S-box aljabar, belum ada kriptanalisis formal, dan kinerja implementasi Python masih rendah. TK-Cipher cocok sebagai sarana belajar, tetapi tidak dimaksudkan untuk menggantikan cipher standar pada sistem nyata.

6.2 Saran

Pengembangan berikutnya sebaiknya memperkuat pembangkitan S-box dan PRNG, melakukan kriptanalisis yang lebih dalam (jumlah S-box aktif, karakteristik diferensial dan linear, serta serangan *related-key*), menyediakan implementasi dengan waktu eksekusi konstan, mengoptimalkan kinerja, mendukung pemrosesan berkas secara *streaming* dan mode AEAD, serta menambahkan penurunan key dari kata sandi.

REFERENSI

- [1] R. Munir, "Konsep block cipher dan mode operasi," slide kuliah IF4020 Kriptografi, Program Studi Teknik Informatika, STEI, Institut Teknologi Bandung, 2026. [Online]. Available: [https://informatika.stei.itb.ac.id/~rinaldi.munir/Kriptografi/2026-2027/08-Block-Cipher-Bagian1-\(2026\).pdf](https://informatika.stei.itb.ac.id/~rinaldi.munir/Kriptografi/2026-2027/08-Block-Cipher-Bagian1-(2026).pdf)
- [2] R. Munir, "Beberapa block cipher (Bagian 1)," slide kuliah IF4020 Kriptografi, Program Studi Teknik Informatika, STEI, Institut Teknologi Bandung, 2026. [Online]. Available: <https://informatika.stei.itb.ac.id/~rinaldi.munir/Kriptografi/2026-2027/09-Beberapa-block-cipher-bagian1-2026.pdf>
- [3] R. Munir, "Beberapa block cipher (Bagian 2)," slide kuliah IF4020 Kriptografi, Program Studi Teknik Informatika, STEI, Institut Teknologi Bandung, 2026. [Online]. Available: <https://informatika.stei.itb.ac.id/~rinaldi.munir/Kriptografi/2026-2027/10-Beberapa-block-cipher-bagian2-2026.pdf>
- [4] R. Munir, "Beberapa block cipher (Bagian 3)," slide kuliah IF4020 Kriptografi, Program Studi Teknik Informatika, STEI, Institut Teknologi Bandung, 2026. [Online]. Available: <https://informatika.stei.itb.ac.id/~rinaldi.munir/Kriptografi/2026-2027/11-Beberapa-block-cipher-bagian3-2026.pdf>

LAMPIRAN

Lampiran A. Tautan Proyek

Item	Tautan
Repositori GitHub	https://github.com/filbertengyo/tk_cipher
GitHub Release	https://github.com/filbertengyo/tk_cipher/releases/latest
API Documentation	https://filbertengyo.github.io/tk_cipher/api/

Lampiran B. Pembagian Tugas

NIM	Nama	Kontribusi
13523126	Brian Ricardo Tamin	Desain awal cipher, operasi ronde dan inverse, CLI dan executable, API docs dan README
13523154	Theo Kurniady	KDF dan CMAC-TK, format file verify-then-decrypt, key schedule, integration dan E2E test
13523161	Arlow Emmanuel Hergara	Padding PKCS#7, mode ECB, CBC, CFB, OFB, CTR, analisis avalanche dan histogram, benchmark
13523163	Filbert Engyo	PRNG TKRand dan S-box dinamis, enkripsi dan dekripsi blok dan test vector, analisis round diffusion dan statistik S-box, analisis entropi dan chi-square

Lampiran C. Data Avalanche Plaintext per Blok

Persentase bit ciphertext yang berubah pada tiap blok ketika 1 bit plaintext dibalik (1000 sampel per mode, plaintext 4 blok).

Mode	Blok	Mean	Min	Max	Std
ECB	0	12,60	0,00	61,72	21,74
ECB	1	13,66	0,00	66,41	22,58
ECB	2	12,07	0,00	64,84	21,41
ECB	3	11,68	0,00	64,06	21,27
ECB	total	12,50	8,98	16,60	1,12
CBC	0	12,11	0,00	64,84	21,54
CBC	1	24,77	0,00	62,50	25,07
CBC	2	36,76	0,00	65,62	22,29
CBC	3	50,07	34,38	64,06	4,54
CBC	total	30,93	8,59	57,03	14,13
CFB	0	0,19	0,00	0,78	0,34
CFB	1	12,49	0,00	62,50	21,51
CFB	2	25,57	0,00	61,72	25,04
CFB	3	37,58	0,78	64,06	21,77
CFB	total	18,96	0,20	44,92	14,08
OFB	0	0,20	0,00	0,78	0,34
OFB	1	0,19	0,00	0,78	0,34
OFB	2	0,20	0,00	0,78	0,34
OFB	3	0,19	0,00	0,78	0,33
OFB	total	0,20	0,20	0,20	0,00
CTR	0	0,20	0,00	0,78	0,34
CTR	1	0,20	0,00	0,78	0,34
CTR	2	0,19	0,00	0,78	0,34
CTR	3	0,20	0,00	0,78	0,34
CTR	total	0,20	0,20	0,20	0,00

Nilai tiap blok dihitung atas seluruh 1000 sampel, termasuk sampel yang bit-nya dibalik di blok lain, sehingga rata-rata per blok pada ECB, CBC, dan CFB lebih rendah dari 50%. Kesimpulan per mode ada pada Tabel 5.1.

Lampiran D. Perintah Reproduksi Analisis

```
uv sync --all-groups
uv run python analysis/avalanche.py # avalanche plaintext dan key, 5 mode
uv run python analysis/round_diffusion.py # justifikasi 16 ronde
uv run python analysis/entropy.py # entropi dan chi-square
uv run python analysis/histogram.py # histogram dan visual ECB vs CBC
uv run python analysis/sbox_stats.py # statistik S-box
uv run python analysis/benchmark.py # throughput dan key setup
uv run python analysis/run_all.py # semua analisis sekaligus
```